

La mort du code manuel

David-Olivier Saban, avec l'aide de Claude Sonnet 4.6 et GPT-5.5

La mort du code manuel

Repenser le SDLC à l'âge des agents

Version : Draft V3

Auteur : David-Olivier Saban, avec l'aide de Claude Sonnet 4.6 et GPT-5.5

Public cible : Engineering Managers, Tech Leads, Heads of Engineering, responsables produit impliqués dans la transformation IA.

Promesse : proposer un modèle d'organisation engineering pour passer d'une adoption individuelle et chaotique de l'IA à un SDLC agentique guidé, mesurable et responsable.

Versions disponibles :

Document	Titre	Usage
Version longue	La mort du code manuel : repenser le SDLC à l'âge des agents	Version détaillée pour Engineering Managers et Tech Leads.
Version management	Repenser l'agilité à l'âge des agents	Version condensée pour management, Heads of Engineering et Product Leaders.

Executive Summary

Le développement logiciel entre dans une nouvelle phase. L'IA ne se limite plus à compléter quelques lignes de code. Les agents peuvent maintenant rechercher, planifier, implémenter, tester, documenter, ouvrir des Pull Requests, assister la review et aider à la livraison. Le centre de gravité du métier se déplace donc de l'exécution manuelle vers l'orchestration du contexte, de la délégation et de la validation.

Cette transformation ne concerne pas seulement la Pull Request. Elle concerne le flux complet du SDLC (Software Development Life Cycle) : idéation, requirements, user stories, implémentation, validation, release et production. Un agent Product Manager (PM) ou Product Design peut aider à transformer une intention en user story. Un agent développeur peut implémenter. Un agent Quality Assurance (QA) peut comparer la story et la delivery, tester la Pull Request, puis valider ou invalider les acceptance criteria. Le code owner reste responsable de la validation technique, mais le flux devient partagé par toute l'équipe.

La thèse de ce document est volontairement directe : l'ère du code manuel comme activité principale du développeur est en train de se terminer pour une large classe de tâches répétables, contextualisables et testables. Cela ne signifie pas que les ingénieurs disparaissent. Cela signifie que leur valeur se déplace. Les meilleurs ingénieurs seront ceux qui savent définir le bon problème, structurer le contexte, découper les missions, superviser plusieurs agents, reviewer avec discernement et assumer les décisions de merge.

Le problème n'est déjà plus seulement technologique. Les modèles progressent vite. Le vrai frein devient organisationnel : documentation éclatée, pipelines trop lentes, QA saturée, review fatigue, prompts non versionnés, managers non formés, backlog pas assez clair, responsabilités mal définies.

L'argument le plus important est le suivant : même si l'IA devait décevoir, les pratiques nécessaires à son adoption amélioreraient déjà l'engineering. Centraliser l'information, versionner les instructions, automatiser les environnements, renforcer la CI/CD (Intégration Continue / Déploiement Continu), clarifier les besoins, raccourcir les boucles de feedback et mesurer le flux sont de bonnes pratiques avec ou sans agents.

Ce whitepaper propose donc un modèle de transformation pragmatique : commencer par le contexte et l'environnement, industrialiser les agents comme une infrastructure, adapter l'organisation, puis guider par des métriques, des audits et de la documentation de référence.

Ce document est la version longue. Une version condensée orientée management, **Repenser l'agilité à l'âge des agents**, reprend les décisions, la roadmap et les messages exécutifs principaux.

Cinq constats

1. L'autocomplétion n'était pas la rupture ; l'agent autonome l'est.

2. La qualité de l'IA dépend d'abord de la qualité du contexte fourni.
3. La vitesse de production de code déplace les goulots vers la review, la QA, la CI/CD et le produit.
4. Les instructions IA doivent être traitées comme du code de production.
5. Tous les rôles peuvent être augmentés, pas seulement les développeurs.

Cinq décisions à prendre

1. Quels types de tâches peuvent être délégués à des agents ?
2. Quel niveau de review est obligatoire selon le risque ?
3. Où vit la documentation produit et technique qui nourrit les agents ?
4. Qui est accountable quand une Pull Request (PR) vient d'un agent ou d'un non-développeur ?
5. Quelles métriques prouvent que l'adoption crée de la valeur ?

Cinq pièges à éviter

1. Chercher le prompt parfait avant de pratiquer.
 2. Confondre adoption d'outil et transformation du SDLC.
 3. Merger plus vite que la capacité de validation.
 4. Laisser chaque équipe réinventer ses propres prompts dans son coin.
 5. Manager la transformation sans avoir soi-même utilisé les agents.
-

Définitions minimales

Avant d’entrer dans le fond, il faut clarifier le vocabulaire. Le mot “IA” est trop large. Il mélange des usages très différents : completion de code, chat conversationnel, agent autonome, script automatisé, workflow CI/CD, assistant de review. Cette confusion ralentit la transformation parce qu’elle permet à chacun de parler d’un objet différent.

Agent : configuration spécialisée qui combine un rôle, des règles de comportement, des interdictions, une mission et une capacité à utiliser des tools ou skills. Un agent développeur ne doit pas se comporter comme un agent PM. Un agent QA ne doit pas avoir les mêmes priorités qu’un agent chargé d’écrire une spécification.

Skill : connaissance réutilisable mobilisable par un ou plusieurs agents. Un skill peut expliquer comment auditer une PR, écrire une user story, maintenir une documentation produit, évaluer des KPIs web ou réaliser une revue de sécurité.

Tool : action déterministe automatisée. Créer un worktree, lancer l’application locale, exécuter la validation, générer un rapport ou démarrer une stack Docker sont des tools. Quand une action doit être exécutée toujours de la même manière, elle ne doit pas dépendre d’une interprétation en langage naturel.

RPI : Research, Plan, Implement, Review. Modèle de travail où l’on sépare explicitement la recherche, la planification, l’exécution et la vérification. Sa valeur est moins dans le nom des étapes que dans la discipline de transfert de contexte.

Context engineering : conception, structuration et maintenance de l’information nécessaire pour qu’un humain ou un agent travaille correctement : documentation, spécifications, décisions, contraintes, exemples, tests, workflows, historique.

ai-artifact : artefact versionné qui formalise la structure IA d’un projet : agents, skills, tools, overlays, références upstream, règles locales et mécanismes d’audit.

Ce document ne dit pas que

Ce document ne dit pas que les développeurs sont inutiles. Il dit que leur centre de gravité change.

Ce document ne dit pas qu'il faut merger sans review. Il dit que la review doit évoluer parce que le volume produit par agents dépasse la capacité de lecture ligne par ligne.

Ce document ne dit pas que l'IA remplace l'architecture. Il dit qu'elle accélère l'exécution et rend les décisions d'architecture encore plus importantes.

Ce document ne dit pas que tous les domaines sont également prêts. Les systèmes critiques, réglementés, legacy ou mathématiquement spécialisés demandent des preuves spécifiques.

Ce document ne dit pas que la gouvernance peut attendre. Il précise toutefois ce que gouverner veut dire ici : guider, informer, aider, supporter, produire de la documentation fiable et pointer vers de bonnes directions. Gouverner ne veut pas dire imposer un framework interne ou une manière unique de penser.

Architecture du document

Partie I : La rupture

Cette partie pose le changement conceptuel : on ne parle plus d'un assistant de code, mais d'un acteur du SDLC.

1. Avant-propos : la vérité inconfortable.
2. Du copilote à l'agent.
3. Le modèle RPI.

Partie II : Le système

Cette partie décrit les fondations techniques et informationnelles nécessaires pour que les agents fonctionnent.

4. L'architecture de l'information.
5. Les instructions IA sont du code de production.
6. Skills, agents et tools comme infrastructure réutilisable.
7. Setup et onboarding de l'agent.

Partie III : L'organisation

Cette partie montre comment les rôles, les équipes et le management changent lorsque l'exécution accélère.

8. Tous les workers, pas seulement les développeurs.
9. La friction comme ennemi principal.
10. Repenser l'organisation sans juste renommer les rôles.
11. Le management comme premier vecteur du changement.

Partie IV : Gouverner et passer à l'échelle

Cette partie transforme le manifeste en programme d'action.

12. Le coût de la transformation et son ROI (Return on Investment).
13. Mesurer ce qui compte.
14. Feuille de route : Discovery, Pilot, Scale, Govern.
15. Conclusion : l'avenir appartient aux orchestrateurs.

Partie I : La rupture

1. Avant-propos : la vérité inconfortable

L'ère du code manuel comme centre de gravité du métier d'ingénieur logiciel est en train de se terminer.

Ce n'est ni la fin du code, ni la fin des ingénieurs, ni la fin du plaisir de construire. C'est la fin d'une organisation du travail où l'écriture manuelle du code était le principal facteur limitant de la création logicielle.

Pendant des décennies, savoir coder vite et bien était le cœur de la valeur. Cette compétence reste utile, parfois critique. Mais sur une part croissante des tâches quotidiennes, la machine exécute plus vite, avec moins de fatigue, et souvent avec un niveau de qualité supérieur lorsqu'elle dispose du bon contexte, des bons tests et des bons garde rails.

La phrase est inconfortable parce qu'elle touche à l'identité. Coder n'a jamais été seulement une activité productive. Coder a été un plaisir, une manière de penser, un espace de concentration, une façon de résoudre des problèmes avec ses mains et sa tête. Il y a dans le code le même plaisir que dans le montage d'un meuble ou la construction d'une maison : le doute, l'effort, le moment où tout s'aligne, puis la satisfaction finale.

Mais quand on peut dessiner le meuble et le recevoir correctement fabriqué, ou quand un architecte et une équipe construisent la maison que l'on a conçue, le plaisir ne disparaît pas. Il change de place. On passe du rôle d'artisan exécutant au rôle de concepteur, d'architecte, de directeur de travaux.

Le développement logiciel vit ce déplacement.

Il restera toujours des cas où l'expertise humaine dépasse la machine. Il restera des domaines où l'IA échoue, où le contexte est trop pauvre, où la stack est trop exotique, où les contraintes sont trop fines pour être absorbées sans supervision. Il restera aussi, au moins pour le moment, la liberté de coder soi-même. On peut encore choisir de ne pas utiliser l'IA pour une tâche qui nous amuse, qui nous apprend quelque chose ou que l'on veut maîtriser de bout en bout.

Mais il faut regarder la tendance en face. Sur une large classe de tâches répétables, contextualisables et testables, beaucoup d'entre nous sommes déjà dépassés par la machine. Je m'inclus dedans. Pas sur tout. Pas tout le temps. Pas sans contexte. Mais suffisamment souvent pour que continuer à organiser le travail comme si rien n'avait changé soit une erreur managériale.

La question n'est donc plus : "l'IA peut-elle aider les développeurs ?". La question est : "comment doit-on organiser le SDLC quand l'exécution devient massivement délégable ?"

Comment donner à l'IA le bon contexte ? Comment éviter la confiance aveugle ? Comment organiser la review quand une personne peut ouvrir vingt Pull Requests dans la journée ? Comment former des PM et des Customer Success Manager (CSM) qui commencent eux-mêmes à faire des Pull Requests ? Com-

ment garder la responsabilité technique quand l'exécution devient largement déléguable ? Comment manager une équipe dont le principal goulot n'est plus l'écriture du code, mais la clarification du besoin, l'architecture de l'information et la qualité des boucles de feedback ?

La bonne nouvelle, c'est que cette transformation est gagnante même dans le scénario pessimiste. Même si l'IA devait décevoir, même si les agents disparaissaient demain, les pratiques nécessaires à leur usage amélioreraient déjà l'engineering : documentation centralisée, instructions versionnées, pipelines CI/CD solides, environnements reproductibles, feedback loops courts, définition plus claire des besoins, responsabilité plus explicite.

L'adoption de l'IA n'est donc pas seulement un pari technologique. C'est un prétexte puissant pour réparer des pratiques que nous aurions déjà dû réparer.

Résultats observés sur mon projet test

Ces chiffres ne doivent pas être lus comme une promesse de vitesse constante. Ils montrent plutôt le changement de régime : lorsqu'une personne sait orchestrer plusieurs agents, outiller son environnement et choisir des tâches réversibles, le système peut absorber ponctuellement des volumes de travail qui auraient été impensables dans un mode purement manuel.

Signal	Observation
Volume de PR exceptionnel	Environ 20 PRs ouvertes dans une même journée : une dizaine par une seule personne, quelques-unes par d'autres collaborateurs, et plusieurs par un bot de mise à jour de dépendances pour réduire les CVEs.
Sort des PRs	Environ la moitié mergée le jour même, 25% abandonnées, 25% reprises ou gérées les jours suivants.
Rythme soutenable	Le rythme réel et durable est plutôt de 1 à 2 PRs par jour et par personne active, pas 20 PRs par jour en continu.
Volume de code exceptionnel	Environ 20 000 lignes produites en moins de 24h pour mettre en place le framework ai-artifacts. Ce volume inclut une part importante de framework, structure, documentation, boilerplate ou code généré, et ne doit pas être comparé à 20 000 lignes de logique métier critique écrites à la main.
Qualité de PR	Les commentaires de review sont passés d'environ une dizaine par PR à souvent 0-2 commentaires sur les PRs bien préparées et bien contextualisées.
Temps de delivery	Exemple color picker CMS (Content Management System) : environ 30 minutes de temps humain cumulé, implémentation agent en 15 minutes, merge en moins d'une heure.

Signal	Observation
Delivery équipe	Sur mon projet test, nous livrons aujourd'hui autant qu'une équipe précédente d'environ douze personnes, avec une équipe beaucoup plus réduite et moins de coordination indirecte.
Coût infra	Capacité de build fortement augmentée sans hausse équivalente des coûts grâce au passage d'une infrastructure fixe à une infrastructure préemptible.
Satisfaction équipe	Satisfaction accrue quand PM/Product Owner (PO)/QA peuvent valider plus tôt sur environnement de PR et participer au flux de delivery.

Avant / après

Avant	Après
Le code est le centre du métier.	Le contexte, la délégation et la review deviennent le centre du métier.
L'IA est une aide ponctuelle.	L'IA devient un acteur du SDLC.
La productivité dépend surtout de la vitesse d'exécution humaine.	La productivité dépend surtout de la qualité du système autour des agents.

Actions concrètes pour un Engineering Manager

1. Arrêter d'évaluer l'IA sur des souvenirs d'outils vieux de six mois.
2. Identifier les tâches quotidiennes où l'équipe exécute encore manuellement par habitude.
3. Choisir un projet pilote et mesurer les frictions réelles au lieu de débattre en théorie.

2. Du copilote à l’agent

Un copilote attend. Un agent agit.

La première erreur consiste à juger les agents actuels avec la mémoire des anciennes expériences d’autocomplétion. Beaucoup d’ingénieurs ont testé des assistants qui proposaient trois lignes de code mal placées, trop lentes, trop tôt ou trop tard. Ces outils étaient irritants parce qu’ils ne comprenaient ni l’intention complète, ni le contexte réel. Ils lisaient une portion de fichier et essayaient de deviner la suite. L’expérience était parfois utile, rarement transformante.

L’autocomplétion était limitée par son contexte. Elle ne savait pas vraiment quand l’idée était complète. Elle ne savait pas si l’utilisateur était en train d’explorer, de corriger ou d’architecturer. Elle voyait rarement plus que le fichier courant ou quelques fragments autour du curseur. Elle était condamnée à produire une aide locale.

La rupture vient de deux choses : la taille du contexte et l’autonomie d’exécution. Avec de grandes fenêtres de contexte, un agent peut tenir en mémoire une partie de la documentation, des instructions claires, une user story, des pièges à éviter, des conventions de projet et des outils disponibles. Avec un environnement bien configuré, il peut aussi planifier, exécuter, tester, corriger, committer, ouvrir une Pull Request, expliquer ce qu’il a fait et demander de l’aide quand il bloque.

Un agent n’est pourtant pas autonome par magie. Il devient utile quand l’environnement le rend autonome : instructions, tools, tests, permissions, feedback loops, définition claire du done. Un agent sans setup est un junior brillant dans une entreprise sans documentation, sans environnement local fiable et sans règles de livraison. Il peut produire quelque chose. Il ne produira pas un système fiable.

La bascule devient évidente quand on arrête de demander à l’IA de “coder une feature” et qu’on commence à lui confier une mission dimensionnée correctement. Un agent ne peut pas implémenter une epic complexe d’un coup. Un humain non plus. La compétence nouvelle consiste donc à découper le travail en missions que l’agent peut résoudre en trente minutes avec un niveau de qualité acceptable.

Ce point change tout. Si la mission est claire, réversible, testable et limitée, l’agent peut livrer vite. Si elle est vague, trop large, mal contextualisée ou chargée de décisions implicites, il va improviser. Et l’improvisation d’un agent à grande vitesse produit du chaos.

Un exemple simple illustre la différence entre outil et agent : “publie mes changements”. Un outil attendrait une commande précise. Un agent correctement configuré crée la branche, vérifie le statut Git, rédige le commit, pousse sur le dépôt distant, prépare le commentaire de Pull Request et signale ce qui reste à vérifier. Il n’y arrive pas toujours parfaitement du premier coup ; il faut l’augmenter avec des skills, des tools et des prompts. Mais la nature de l’interaction à change.

La confiance devient asymétrique. On ne fait pas confiance à l’agent pour tout.

On lui fait confiance pour certains types de tâches : fixes simples suffisamment documentés, génération de tests, production de documentation lorsque le contexte est donné, migrations de frameworks, travail répétitif, changements dans des zones que l'on maîtrise mal mais que l'agent peut parcourir systématiquement. En revanche, on supervise les Pull Requests, on teste systématiquement, on relit les cas de tests et on garde un regard particulier sur les fichiers critiques.

Le risque n'est pas de déléguer. Le risque est de déléguer sans système de contrôle.

La vitesse crée un problème nouveau. Une personne peut produire ou faire produire un volume de changements impossible à reviewer ligne par ligne dans un temps raisonnable. Dans un cas concret, la mise en place d'un framework ai-artifacts a représenté environ 20 000 lignes en moins de 24h. Ce chiffre doit être interprété correctement : il ne s'agissait pas de 20 000 lignes de logique métier critique écrites à la main, mais d'un ensemble de structure, framework, documentation, boilerplate et code généré. Le problème reste le même : le volume dépasse rapidement la capacité de review exhaustive.

Il faut donc choisir ses batailles : fichiers critiques, changements dangereux, outputs validables par tests, génération mécanique, découpage des gros changements.

L'Engineering Manager doit comprendre qu'il ne juge pas "l'IA". Il juge une version spécifique, dans un setup spécifique, avec un contexte spécifique. Comparer les agents actuels aux assistants d'il y a un an revient à comparer un smartphone moderne aux premiers téléphones portables. Les premiers permettaient de téléphoner dans de bonnes conditions. Les seconds ont changé la manière de vivre. La différence est que ce saut n'a pas pris vingt ans. Il a pris quelques mois.

Avant / après

Avant	Après
L'assistant propose du code.	L'agent exécute une mission.
Le développeur garde tout le contexte en tête.	Le contexte est donné explicitement à l'agent.
La review suit une production humaine lineaire.	La review doit absorber une production parallèle et massive.

Actions concrètes

1. Remplacer les prompts vagues par des missions de trente minutes maximum.
2. Définir les zones où l'agent peut agir seul et les zones qui demandent supervision forte.

3. Construire une checklist de review adaptée au volume : fichiers critiques, tests, migrations, sécurité, effets de bord.
-

3. Le modèle RPI

Le RPI n'est pas une cérémonie. C'est un protocole de transfert de contexte.

Research, Plan, Implement, Review. Le modèle semble simple, presque banal. Sa valeur ne vient pas du nom des étapes, mais de la discipline qu'il apporte : séparer la recherche, la planification, l'exécution et la vérification, puis rendre chaque phase explicite pour les humains et les agents.

Cette intuition rejoint deux approches publiées par les grands fournisseurs cloud. Microsoft HVE Core structure le travail agentique autour de prompts RPI réutilisables. AWS, de son côté, parle d'AI-Driven Development Life Cycle (AI-DLC) et décrit un modèle où l'IA crée des plans, pose des questions de clarification et implémente après validation humaine. Les vocabulaires diffèrent, mais le point commun est le même : l'IA n'est plus seulement un assistant local, elle devient un participant du cycle de développement, sous supervision humaine.

Avant, le flux était souvent flou. Un PM arrivait avec une feature écrite seul, parfois vague, parfois déjà chargée d'une solution technique implicite, souvent sans exemples fonctionnels solides. L'équipe passait un temps considérable à clarifier le besoin : quel est le problème business ? Quels use cases couvre-t-on ? Quels edge cases ignore-t-on volontairement ? Puis le développement commençait et l'on découvrait que des pans entiers de la spécification avaient été ratés. Il fallait revenir au PM, puis au design, puis à l'architecture. Des jours disparaissaient dans une succession d'approximations.

Le changement important n'est pas d'avoir collé l'étiquette RPI sur le processus. Le changement important est la restructuration du transfert d'information entre PM, Tech Lead, équipe et agents.

Dans le nouveau flux, le PM fait un vrai travail de recherche en amont : besoins clients, irritants, opportunités, priorités. Il utilise des agents spécialisés pour transformer cette recherche en documents fonctionnels compréhensibles : pourquoi cette feature, quel problème elle résout, quels scénarios elle couvre, quels critères d'acceptation permettent de la valider. Le Tech Lead fait le même travail côté technique : options d'architecture, risques, dépendances, contraintes de déploiement, stratégies de réversibilité.

Ces documents ne restent pas dans une conversation éphémère. Ils entrent dans la documentation produit. Ils deviennent une base de travail pour les humains et pour les agents.

La boucle de refinement change alors de nature. Plusieurs fois par semaine, l'équipe commente les stories, corrige les approximations, décide ce qui doit être pris par un humain et ce qui peut être délégué à un agent autonome. Le Research et le Plan restent fortement humains, mais ils sont augmentés par les agents. L'Implement est souvent délégué. Le Review redevient humain, assisté par des agents et par l'automatisation.

AWS AI-DLC insiste sur ce même point avec la notion d' "AI Powered Execu-

tion with Human Oversight” : l’IA peut produire des plans et exécuter, mais les décisions critiques restent humaines, parce qu’elles demandent le contexte business et organisationnel. C’est exactement la frontière que le RPI doit rendre explicite.

La décision de déléguer suit quelques questions simples : quel est l’impact du changement ? Est-il réversible ? Peut-on le livrer sans dommage ? Est-il backward compatible ? Peut-on limiter son impact en production ? Les décisions réversibles avec fort ROI et faible impact sont des candidates naturelles à la délégation.

Un exemple concret : ajouter un color picker dans un CMS. Ce n’était pas une feature majeure, mais justement un bon exemple de changement produit qui aurait facilement pu traîner dans un backlog. Le découpage réel a été le suivant : environ dix minutes pour écrire la story, dix minutes pour le PM afin de déléguer la tâche à un agent et valider fonctionnellement le résultat, quinze minutes pour l’agent afin de coder le changement, puis dix minutes pour le développeur afin de reviewer et merger. Au total : environ trente minutes de temps humain cumulé, et un merge en moins d’une heure.

Ce type de changement n’est pas spectaculaire techniquement. Il est important organisationnellement. Il montre que les petites améliorations de qualité de vie produit peuvent être traitées au moment où elles sont utiles, sans attendre qu’un développeur veuille bien les prendre entre deux sujets plus prioritaires.

Mais le RPI ne supprime pas le jugement humain. Les agents prennent des raccourcis. Ils affirment parfois avoir fait quelque chose sans l’avoir vraiment fait. Ils peuvent interpréter un bug de manière trop locale.

Un cas typique : une UI qui ne sauvegardait pas correctement la position d’écran entre deux pages. L’agent autonome a compris le problème comme un cas spécifique de bouton retour, alors que le bug était global. Après plusieurs tentatives, une reformulation humaine simple a débloqué la situation : pourquoi une hashtable ne suffirait-elle pas à mémoriser la position ? L’agent a alors expliqué les limitations du framework et trouvé une solution.

La compétence humaine irremplaçable n’était pas l’expertise technique pure. C’était le bon sens, la reformulation et le recul.

Le RPI court-circuite quand la vitesse dépasse la capacité de vérification. Une journée à environ 20 PRs en donne une illustration utile : une dizaine venaient d’une seule personne qui avait traité plusieurs bug fixes, quelques-unes venaient d’autres collaborateurs, et plusieurs étaient générées par un bot de mise à jour de dépendances pour éviter des CVEs. Environ la moitié a été mergée le jour même, un quart a été abandonné, et un quart a été repris ou géré les jours suivants. Ce n’est pas un rythme durable ; le rythme soutenable est plutôt de 1 à 2 PRs par jour et par personne active. Mais cette pointe montre que le système de validation peut être saturé très vite.

Ces situations ne sont pas des accidents individuels. Elles révèlent un décalage structurel : la production accélère plus vite que le système de validation.

Matrice de délégation

Type de tâche	Agent	Humain	Validation recommandée
Boilerplate	Fort	Faible	Tests automatisés + review rapide.
Documentation	Fort	Moyen	Review métier ou technique selon sujet.
Bug simple non critique	Fort	Moyen	Test de reproduction + test de non-regression.
Refactoring local	Fort	Moyen	Tests + inspection fichiers critiques.
Migration framework	Moyen/Fort	Fort	Plan de rollback + CI complète + review experte.
Architecture structurante	Moyen	Fort	ADR + revue tech lead/architecte.
Bug prod critique	Faible/Moyen	Fort	Pair review + validation manuelle + rollback plan.
Securite	Moyen	Fort	Tooling security + expert review.
Performance complexe	Moyen	Fort	Profiling + benchmark + review spécialisée.
Tache produit vague	Faible	Fort	Clarification Research/Plan avant délégation.

Avant / après

Avant	Après
La spécification est un document imparfait transmis au dev.	La spécification devient un contexte vivant, versionné et utilisable par agents.
Le développeur clarifié pendant l'implémentation.	L'équipe clarifié avant, puis l'agent exécute dans un cadre limite.
La review arrive à la fin comme filet de sécurité.	La review est intégrée au système : tests, agents, pairs, CI/CD, code owners.

Actions concrètes

1. Forcer chaque feature à produire un artefact Research et un artefact Plan avant implémentation.
 2. Classer les tâches selon réversibilité, impact prod et niveau de contexte disponible.
 3. Instaurer des checkpoints courts pendant l'exécution agentique, au lieu d'attendre la PR finale.
-

Partie II : Le système

4. L'architecture de l'information comme fondation

Si tu ne peux pas donner le contexte à une IA, tu ne peux probablement pas onboarder correctement un nouvel ingénieur.

L'IA ne crée pas le problème de documentation. Elle le rend impossible à ignorer. Les organisations qui fonctionnaient grâce à des conversations, des habitudes, des experts historiques et des documents éparpillés découvrent que cette connaissance tacite ne scale pas. Les agents ne peuvent pas deviner durablement ce que l'entreprise n'a pas rendu explicite.

Le problème est connu : Confluence, wikis internes, PowerPoints sur SharePoint, documents sur laptops, discussions Teams, tickets obsolètes, schémas jamais mis à jour, décisions orales. Trouver toutes les informations liées à un produit relève souvent du miracle. Les humains s'en accommodent mal. Les agents ne s'en accommodent pas du tout.

Quand le contexte est dispersé, l'agent compense. Il hallucine, invente des conventions, modifie ce qu'il ne devrait pas modifier, ignore des contraintes non documentées. Le problème n'est pas seulement que l'IA est mauvaise. Le problème est que l'organisation demande à une machine de deviner ce qu'elle-même n'a jamais pris la peine d'écrire.

La réponse est une philosophie simple : tout ce qui structure le produit doit être versionné, accessible et proche du travail réel. Quand c'est possible, cela veut dire dans le même repository Git. Le monorepo n'est pas seulement une préférence technique ; c'est une architecture de vitesse.

Position claire : pour un produit comme celui-ci, je recommande le monorepo. Plus largement, je recommande tout ce qui réduit la surface de synchronisation : monorepo quand c'est possible, trunk-based development quand l'organisation sait le tenir, petites PRs, feature flags, validation automatisée, environnements de PR et ownership clair. L'objectif n'est pas de défendre une religion d'architecture. L'objectif est de limiter le nombre d'endroits où l'information diverge et où le flux ralentit.

Infrastructure as code. Documentation as code. Specification as code. Code applicatif. Schemas de base de données. Pipelines. Décisions d'architecture. Instructions IA. Tout ce qui peut vivre dans le même repository devrait y vivre, parce qu'un changement produit traverse rarement une seule couche. Une feature peut exiger une modification UI, une évolution de schema, un ajustement CI/CD, une mise à jour de documentation, un changement d'infrastructure et une instruction agent. Les séparer par défaut ralentit le système.

Quand plusieurs repositories sont inévitables, il faut réduire leur nombre au minimum, les référencer explicitement entre eux, et s'assurer que les agents ont accès au même niveau d'information que les humains. Sinon l'organisation paie un coût caché : multiplication des agents, multiplication des cross-références, dilution de l'information, redondance documentaire, ralentissement du flux de

développement, complexification des interactions entre repositories et équipes.

Le problème n'est pas seulement la navigation. C'est la divergence. Plus les repositories se multiplient, plus il faut maintenir des agents similaires, des instructions similaires, des workflows similaires et des documentations similaires dans des espaces différents. La probabilité de divergence augmente avec le nombre de repositories. Une règle mise à jour ici ne l'est pas là. Une décision d'architecture est référencée dans un projet mais pas dans l'autre. Un agent travaille avec une information obsolète parce que le repo qu'il consulte n'est pas la source la plus récente.

Le monorepo ne résout pas tout, mais il réduit la surface de synchronisation. Il permet de garder ensemble les changements qui doivent être compris ensemble. Il permet aux agents d'avoir une vision plus complète du système. Il limite les erreurs de documentation, réduit les cross-références fragiles et conserve la vélocité globale.

L'exemple de mon projet test est parlant. Nous fonctionnions déjà en monorepo pour le code applicatif, puis nous avons progressivement rapproché la CI/CD, l'infrastructure, la documentation, le CMS, la UI et les tests End-to-End (E2E) dans le même repository quand cela avait du sens. Si chacun de ces éléments avait eu son propre repository, il aurait fallu maintenir jusqu'à six repositories : six documentations, six README, six structures agentiques, six pipelines, potentiellement six commits pour un seul changement transverse, et jusqu'à six équipes ou ownerships à coordonner. Le coût de synchronisation aurait explosé avant même que l'IA entre en jeu.

Dans un contexte agentique, ce coût devient encore plus dangereux. L'agent doit comprendre le système pour proposer un changement cohérent. Si le CMS, la UI, les tests, l'infrastructure, la documentation et la CI/CD sont séparés, il faut lui redonner artificiellement une vision d'ensemble que le repository aurait pu lui offrir naturellement. Chaque séparation devient un contexte à reconstruire.

Cette proximité de l'information permet un changement fondamental : une même Pull Request peut modifier une feature, ajuster l'interface, mettre à jour la documentation, adapter un schema et modifier la CI/CD si nécessaire. Non pas pour supprimer la revue, mais pour éviter les synchronisations ad hoc entre cinq changements séparés. Les code owners gardent leur rôle. Ils valident ce qui relève de leur expertise. Mais ceux qui comprennent la chaîne de bout en bout peuvent proposer un changement cohérent de bout en bout.

La séparation of concerns reste un principe utile. Mais elle devient dangereuse quand elle sert de justification à la fragmentation de l'information. Découper pour découper ne fait pas gagner du temps. Cela déplace le coût vers la recherche, la coordination et la reconstitution mentale du système.

Un humain augmente et une IA augmentée ont besoin de la même chose : une vue d'ensemble. Le contexte explicite devient un actif stratégique. Il ne sert pas seulement à faire travailler la machine. Il sert à former les nouveaux arrivants, à

réduire la dépendance aux experts historiques, à rendre les décisions auditable, à accélérer les raffinements, à améliorer la qualité des PRs.

Le context engineering n'est donc pas une mode. C'est une extension naturelle de l'architecture logicielle. On n'architecture plus seulement le code. On architecture l'information nécessaire pour faire évoluer le code.

Avant / après

Avant	Après
La connaissance vit dans des silos.	La connaissance est versionnée, référencée et, quand possible, colocated dans le même repository.
L'onboarding dépend des personnes disponibles.	L'onboarding s'appuie sur un contexte explicite.
L'agent devine les contraintes.	L'agent lit les contraintes.
Les repositories se multiplient par domaine.	Les repositories sont limités au minimum et cross-références explicitement.
Le branching compense la peur de casser.	Trunk-based development, petites PRs et feature flags réduisent la durée de divergence.

Actions concrètes

1. Cartographier les sources d'information produit et identifier celles qui sont cachées ou obsolètes.
2. Déplacer progressivement spécifications, décisions et documentation critique vers un format versionné.
3. Adopter un monorepo pour regrouper les artefacts as-code quand c'est possible, ou documenter explicitement les références entre repositories.
4. Réduire la durée de divergence avec trunk-based development, petites PRs, feature flags et environnements de PR.
5. Tester l'onboarding d'un agent comme test de qualité de l'onboarding humain.

5. Les instructions IA sont du code de production

Un prompt non versionné est de la dette technique.

Beaucoup d'équipes commencent avec un fichier unique : `AGENTS.md`, parfois 600 lignes ou plus. Et encore, c'est le meilleur des cas. La majorité n'a même pas ce fichier. Tout est dans les habitudes, les conversations, les prompts personnels, les essais de chacun. Le résultat est prévisible : agents inconsistants, instructions ignorées, tests oubliés, worktrees non créés, commits trop tôt, PRs à moitié implémentées, frustration générale.

Un exemple concret illustre ce risque mieux que n'importe quelle théorie. GMS Runner est un projet open-source interne chez Amadeus, ouvert à toutes les contributions, qui suit les bonnes pratiques de code habituelles de l'organisation. Quand j'ai voulu utiliser un agent pour corriger un bug sur ce projet, il n'existait aucune configuration agentique : pas d'`AGENTS.md`, pas de règles de workflow, pas de guardrails.

L'agent a travaillé de manière "naturelle" : il a analysé le code, estimé avoir identifié la cause du bug, produit une correction, puis commité et poussé le changement sans que je lui demande, et sans vérifier que le fix fonctionnait réellement. J'ai eu beau lui répéter les consignes — ne pas committer sans confirmation explicite, vérifier d'abord que le bug est reproduit et corrigé — l'agent continuait d'oublier ces règles à chaque itération, tout en s'excusant poliment. La même instruction, répétée dans la conversation, n'avait aucun effet durable.

Le problème n'était pas le modèle. C'était l'absence d'instructions versionnées.

Le jour où j'ai créé un `AGENTS.md` décrivant la méthodologie de travail — Test-Driven Development (TDD) et Acceptance Test-Driven Development (ATDD), pas de commit sans confirmation explicite de ma part, vérification du fix avant toute autre action — le comportement a radicalement changé. L'agent a commencé par créer un test pour reproduire le bug. Il a ensuite corrigé le code, vérifié que le test passait, puis s'est rendu compte que le fix était incomplet. Il a itéré, rajouté des tests unitaires et des tests E2E dans un projet qui en manquait, et n'a commité aucune ligne avant que je valide explicitement.

La même tâche. Le même agent. Le même modèle. La seule différence : un fichier d'instructions versionné qui décrivait le workflow attendu.

Le problème n'est pas que les agents sont incapables de suivre des instructions. Le problème est que les instructions sont souvent mal conçues, non composées, non testées et non gouvernées.

Traiter les instructions IA comme du code de production signifie les versionner, les factoriser, les composer, les relire, les auditer et supprimer ce qui est obsolète. Cela oblige aussi à distinguer les règles universelles du contexte projet, puis à séparer ce qui relève d'un agent, d'un skill ou d'un tool.

L'idée d'un **ai-artifact** va dans ce sens : formaliser la structure IA d'un projet et référencer des documents ou méthodes réutilisables. Personne ne maintiendra seul les meilleurs prompts du monde. Les fournisseurs, les chercheurs, les grandes communautés open-source et les meilleurs praticiens produiront des configurations de référence. La bonne architecture doit permettre de les utiliser sans lock-in, puis d'ajouter une couche spécifique au projet.

Je propose ce framework **ai-artifacts** pour gérer cette partie : mettre à disposition des bases de connaissances, des exemples d'agents, des skills, des workflows et des références versionnées dans lesquelles les équipes peuvent piocher. Il ne s'agit pas d'imposer un kit complet à toutes les équipes, mais de leur donner une bibliothèque d'inspiration fiable, auditable et composable.

La structure cible est simple : une base upstream solide, des add-ons pour les besoins vraiment spécifiques, des overlays pour adapter un skill ou un agent existant, et un fichier de règles projet qui ne contient que ce qui doit absolument être respecté.

Un point est non négociable : cette structure ne doit pas devenir un standard interne imposé par le management. La gouvernance doit guider, pas contraindre. Les standards utiles s'imposent de facto parce qu'ils sont adoptés massivement, parce qu'ils ont prouvé qu'ils étaient meilleurs, plus simples, plus fiables ou plus faciles à maintenir. Ils ne deviennent pas bons parce qu'une organisation centrale les a déclarés obligatoires.

AGENTS.md ne doit pas devenir une encyclopedie. Il ne sert à rien d'expliquer à un agent moderne toutes les bonnes pratiques de code. Il les connaît souvent mieux que nous. On peut lui rappeler des principes : respecter l'ACIDite, appliquer SOLID, suivre Tidy First, penser à Kent Beck, Martin Fowler, Robert Martin. Mais il faut éviter de re-documenter ce que le modèle sait déjà.

La règle d'or est simple : documenter ce que l'agent ne peut pas savoir seul, c'est-à-dire le contexte local, les contraintes spécifiques, les décisions d'architecture, les workflows internes, les commandes exactes et les interdits non négociables.

De 600 à 70 lignes : ce que ce chiffre révèle

Un **AGENTS.md** de 600 lignes échoue pour une raison précise : l'information importante est noyée dans du contenu que le modèle connaît déjà. Plus le fichier est long, plus l'agent perd le signal critique dans le bruit. Il lit, il traite, puis il oublie — pas par incompetence, mais parce que la densité d'information utile est trop faible pour que les règles vraiment importantes émergent durablement à chaque interaction.

Six cents lignes représentent facilement environ trente pages d'instructions. La métaphore humaine est utile ici, même si elle reste une simplification : on ne s'attendrait pas à ce qu'un nouvel arrivant respecte à la lettre trente pages d'instructions détaillées tout en réalisant correctement une tâche complexe. Il ne faut pas attendre cela d'un agent non plus. Plus on empile les règles, plus on transforme l'instruction en bruit.

J'ai réduit mon propre AGENTS.md de 600 à 70 lignes. Le résultat n'est pas un agent appauvri. C'est un agent qui fait moins d'erreurs. Il n'a plus besoin de s'excuser d'avoir oublié de créer un worktree avant d'implémenter une feature, d'appliquer le TDD, ou de valider les changements avant de committer et pusher. Ces règles sont respectées parce qu'elles sont maintenant les seules règles présentes — claires, sans bruit, sans compétition avec cinquante autres paragraphes de moindre importance.

Que contenaient les 530 lignes supprimées ? Essentiellement trois catégories :

- **Les bonnes pratiques générales** que le modèle connaît déjà : SOLID, clean code, gestion des erreurs, sécurité de base. Les ré-expliciter ne fait que diluer les règles spécifiques au projet.
- **Les explications pédagogiques** sur le pourquoi de chaque règle. L'agent n'a pas besoin de comprendre le raisonnement pour respecter le garde-rail.
- **Les scénarios hypothétiques** jamais rencontrés, ajoutés par précaution mais jamais activés.

Ce qui reste dans les 70 lignes : le workflow obligatoire (TDD, worktree, validation avant commit), les interdits non négociables, les commandes exactes spécifiques au projet, et des pointeurs vers les skills et tools qui gèrent le reste.

C'est précisément la décomposition en agents, skills et tools qui rend cette réduction possible. Les règles de comportement restent dans l'agent. La méthodologie de travail devient un skill réutilisable. Les actions répétables deviennent des tools déterministes. L'AGENTS.md n'a plus besoin de tout contenir — il délègue.

Le bénéfice est immédiat et mesurable : moins d'erreurs, moins d'excuses, moins de temps perdu à corriger des oublis, plus de confiance dans le résultat. C'est un cercle vertueux, et il commence par supprimer du texte.

Cette industrialisation rencontre une résistance. Certains pensent que chaque agent sera tellement spécifique à une industrie, un client ou un projet qu'aucun framework n'est possible. C'est partiellement vrai. Mais si l'on peut écrire des livres sur la programmation, l'Agile où l'architecture, c'est qu'il existe des principes généraux. Ces principes doivent être publiés, référencés, réutilisables. Le contexte vient ensuite.

L'objectif n'est pas de créer une usine à gaz propriétaire, ni une doctrine interne unique. L'objectif est de proposer des références fiables, des exemples solides, des composants réutilisables et une structure simple, extensible, supprimable le jour où une meilleure solution arrive. La valeur n'est pas dans le verrouillage. Elle est dans l'aide apportée aux équipes.

Anti-patterns

Anti-pattern	Symptôme	Correction
Prompt spaghetti	Instructions longues, contradictoires, non maintenues.	Factoriser en agents, skills, tools.
Règles non testées	L'agent affirme avoir respecté le process mais ne l'a pas fait.	Ajouter des tools de validation et des checkpoints.
Documentation inutile	Le prompt expliqué des bonnes pratiques générales que le modèle connaît déjà.	Garder uniquement le contexte local et les interdits.
Framework interne verrouillant	Tout le monde dépend d'une solution propriétaire impossible à remplacer.	Préférer une structure simple, composable, supprimable.
Gouvernance prescriptive	Une équipe centrale imposé un framework et une méthode unique.	Guider, documenter, supporter ; laisser les meilleurs standards s'imposer par adoption.

Avant / après

Avant	Après
Les prompts vivent dans les conversations.	Les instructions sont versionnées et auditées.
AGENTS.md devient un fourre-tout.	AGENTS.md contient les règles critiques et référence le reste.
Chaque équipe reinvente ses prompts.	Les équipes composent une base partagée avec des overlays locaux.

Actions concrètes

1. Extraire les instructions critiques des conversations et les versionner.
2. Supprimer des instructions tout ce que le modèle sait déjà, sauf rappel court et intentionnel.
3. Mettre en place un audit régulier des agents, skills, prompts et tools.

6. Skills, agents et tools comme infrastructure réutilisable

Un agent sans skills ni tools est un junior brillant sans environnement de travail.

Pour industrialiser l'usage des agents, il faut une taxonomie claire. Sinon tout devient prompt. Et quand tout devient prompt, plus rien n'est vraiment maintenable.

Un agent est un rôle avec des règles. Un agent développeur suit un workflow de code, respecte des guardrails, sait comment tester, quand demander de l'aide, comment ouvrir une PR. Un agent PM parle de manière fonctionnelle, évite les termes techniques inutiles, clarifié le problème client, les critères d'acceptation et les impacts produit. Un agent Product Designer se concentre sur le problème utilisateur, l'intention design, les principes d'interaction et les critères vérifiables par un designer.

Un skill est une connaissance réutilisable. Comment maintenir la documentation interne. Comment auditer le code. Quels sont les grands KPIs d'une application web. Comment écrire une story. Comment conduire une revue de sécurité. Le skill n'est pas une personne. C'est une capacité que plusieurs agents peuvent mobiliser.

Un tool est une action déterministe. Créer un worktree. Lancer l'application en local. Démarrer Docker et les services nécessaires. Exécuter la validation. Générer un rapport. La valeur du tool est de limiter la variabilité. Quand une action doit être répétée toujours de la même manière, elle ne doit pas être décrite dans un prompt. Elle doit être automatisée.

Cette séparation évite un piège fréquent : mettre trop de choses dans les instructions. Si un agent doit créer un worktree, il ne doit pas interpréter quinze lignes de procédure. Il doit appeler un outil. Si une équipe répète les mêmes instructions de review, elle doit créer un skill. Si un comportement est non négociable, il appartient aux règles de l'agent.

Le skill le plus précieux n'est pas toujours celui qui produit le plus de code. C'est souvent le skill d'audit. Il vérifie que les skills restent compacts, composables et non redondants. Il détecte les actions déterministes qui devraient être des tools. Il signale les instructions obsolètes. Il joue pour l'infrastructure IA le rôle que les tests et linters jouent pour le code.

Sans audit, une bibliothèque de skills se dégrade silencieusement. Les doublons apparaissent. Les règles se contredisent. Les agents suivent des chemins divergents. Les prompts grossissent. Les équipes perdent confiance.

Il reste une zone immature : l'obsolescence. Quels skills sont encore utilisés, par qui, avec quel résultat ? Lesquels dérivent de leur version upstream ? Quels agents ignorent leurs propres guardrails ? Il faut construire un audit trail. L'infrastructure IA aura besoin de monitoring comme n'importe quelle infrastructure critique.

Exemple : agent Product Designer

La PR #993 ajoute un agent **pd-feature-writer**. Son rôle n'est pas de coder. Il aide un Product Designer à produire des issues claires, concises, compréhensibles par un designer, un PM et un lecteur non natif. Il privilégie le Globish, évite les détails d'implémentation, se concentre sur les problèmes utilisateurs, les objectifs produit, les objectifs design, les principes d'interaction, les contraintes et les limites de scope.

Cet exemple est important : l'agent n'est pas seulement un accélérateur de code. Il est une manière pour chaque rôle d'encoder sa propre méthode de travail dans un artefact réutilisable. Le PM ou le designer ne demande plus seulement à l'équipe de mieux écrire les issues. Il formalise ce que "mieux écrire" veut dire.

Exemple : agent QA

L'agent QA est né d'un constat partagé avec l'équipe MSFT lors d'une analyse du flux. Les agents développeurs produisaient du code fonctionnellement correct, mais omettaient régulièrement certains éléments des spécifications décrits dans les user stories. Plutôt que de renforcer la boucle interne de l'agent qui code, nous avons convenu qu'il était plus pertinent d'introduire un acteur externe : un agent QA dont le rôle est de comparer ce qui était demandé et ce qui a été livré, une fois la PR compilée.

Pour fonctionner, cet agent a besoin de conditions préalables non négociables :

1. Toutes les stories doivent avoir une Definition of Done (DOD) claire.
2. Des acceptance criteria explicites et vérifiables.
3. Des cas de validation décrits dans la user story.
4. Une liste d'éléments que le QA doit toujours vérifier, quelles que soient les AC spécifiques.

L'agent QA est un agent fonctionnel, pas technique. Il ne lit pas le code. Il utilise Playwright pour interagir avec l'environnement de PR comme le ferait un testeur humain, en s'appuyant uniquement sur la documentation et les spécifications. Il n'a pas besoin de comprendre l'implémentation pour valider que le comportement attendu est présent.

Son output est structuré : un tableau récapitulatif des AC passés ou échoués, accompagné d'un screenshot comme preuve pour chaque critère. Ce niveau de traçabilité est intentionnel — il augmente la confiance de l'équipe dans le résultat et rend la validation visible pour le PM, le PO et le code owner sans qu'ils aient besoin de re-tester manuellement.

Enfin, si l'agent juge que les AC ne couvrent pas suffisamment la spec ou la documentation produit, il conduit quelques tests exploratoires supplémentaires, signale les écarts et les inclut dans son rapport.

Pour que ce modèle soit systématique, nous avons modifié la pipeline. Une fois l'environnement de PR déployé, la CI/CD ajoute automatiquement un commentaire sur la PR contenant les liens testables vers toutes les surfaces de

l'environnement. Un workflow détecte la présence de ce commentaire et déclenche l'agent QA. L'agent sait ainsi qu'il peut commencer ses validations sans attendre d'instruction manuelle. Le déclenchement est événementiel, non pas planifié : chaque PR compilée et déployée reçoit automatiquement sa validation fonctionnelle.

Ce modèle illustre un principe clé : l'agent QA ne remplace pas la QA. Il déplace le moment de détection des manquements de "après merge" à "avant merge", ce qui raccourcit significativement la boucle de correction. Il rend aussi les développeurs et les agents plus responsables de la qualité de leur livraison, parce qu'ils savent qu'un regard fonctionnel indépendant viendra comparer la story et la PR.

Avant / après

Avant	Après
Tout est prompt. L'agent improvise les procédures. Les skills s'empilent sans gouvernance.	Les rôles, connaissances et actions sont séparés. Les tools exécutent les procédures déterministes. Les skills sont audités comme du code.

Actions concrètes

1. Classer chaque instruction existante : règle d'agent, skill réutilisable ou tool déterministe.
2. Automatiser les procédures répétables au lieu de les décrire en langage naturel.
3. Créer un audit trail minimal : skill appelé, agent utilisateur, résultat, date, dérive éventuelle.

7. Setup et onboarding de l'agent

Un agent doit être onboardé comme un newcomer.

Une erreur fréquente consiste à croire que l'agent échoue parce qu'il est mauvais, alors qu'il échoue parce qu'on l'a placé dans un environnement où aucun nouvel arrivant ne pourrait travailler correctement. Un développeur humain reçoit un laptop, des accès, des explications, un environnement local, des commandes, des tests, des collègues à qui poser des questions, des documents, des habitudes implicites. L'agent a besoin de l'équivalent, mais sous forme explicite et exécutable.

La mise en place est donc une phase à part entière. Elle ne se résume pas à écrire un bon prompt. Il faut fournir à l'agent un environnement de développement fonctionnel, une machine capable d'accéder à internet, assez de ressources pour faire tourner le projet en local, un browser Playwright qu'il sait utiliser, des tests E2E stables, une documentation à jour, des outils MCP (Model Context Protocol) pour accéder à GitHub ou Figma, une gestion correcte de sa mémoire, des tools déterministes et des instructions claires.

Tout ce qui est de facto disponible pour un développeur doit être rendu explicite pour l'agent. Si l'environnement local est fragile, si la documentation est obsolète, si les tests ne sont pas fiables, si les accès sont incomplets, l'agent ne fera qu'amplifier ces problèmes. L'IA ne remplace pas l'onboarding. Elle en révèle la qualité réelle.

AWS AI-DLC formule un principe proche lorsqu'il explique que l'IA conserve et maintient le contexte persistant du projet en stockant plans, exigences et artefacts de design dans le repository. C'est un point essentiel : l'agent ne doit pas seulement recevoir du contexte dans une conversation. Il doit retrouver ce contexte dans le projet, d'une session à l'autre, comme un membre de l'équipe retrouverait la documentation et l'historique.

Cette phase est aussi psychologiquement coûteuse. Pour calibrer les agents, il faut parfois prendre la même tâche et la faire développer encore et encore jusqu'à obtenir un résultat satisfaisant. La même feature peut être livrée vingt fois dans la même semaine, non pas parce que l'équipe veut la livrer vingt fois, mais parce qu'elle teste la méthode, les instructions, les guardrails, les tests et les critères de done.

Il faut toutefois éviter l'erreur inverse : vouloir construire tout le setup agentic avant de commencer. Les agents, skills et tools utiles ne se devinent pas tous à l'avance. Ils émergent au fil des échecs, des frictions et des besoins réels. La première phase ne doit donc pas être un grand chantier de prompts. Elle doit surtout préparer le terrain non-IA : environnement local reproductible, CI/CD stable, environnements de PR, documentation centralisée, tests E2E fiables même peu nombreux, accès GitHub/Figma/documentation/métriques, et rapprochement des repositories connexes quand ils bloquent le flux.

Ce travail peut sembler absurde vu de l'extérieur. Il ne l'est pas. Chaque répétition permet d'identifier une faille différente : une instruction ambiguë, un test manquant, un tool trop variable, une documentation insuffisante, un accès absent, une limite de mémoire, une mauvaise définition du done. La répétition n'est pas du gaspillage si elle transforme un échec en infrastructure réutilisable.

Le setup agentique est donc un investissement. Il ressemble moins à une configuration d'outil qu'à la construction d'un environnement d'apprentissage et de delivery. Au début, on ralentit pour comprendre pourquoi l'agent échoue. Ensuite seulement, on accélère. La vitesse ne doit pas être l'objectif initial d'un pilot ; elle devient une conséquence lorsque le système fonctionne.

Avant / après

Avant	Après
L'agent est lancé avec un prompt et de l'espoir.	L'agent reçoit un environnement, des tools, des tests, des accès et du contexte.
Les échecs sont interprétés comme limites du modèle.	Les échecs servent à identifier ce qui manque dans le setup.
La même tâche rate plusieurs fois.	Chaque répétition améliore instructions, guardrails, tests ou tools.

Actions concrètes

1. Onboarder les agents comme des newcomers : environnement, accès, tests, documentation, tools.
2. Transformer les échecs répétés en améliorations de setup plutôt qu'en jugements sur le modèle.
3. Stabiliser les tests et tools avant de chercher à augmenter l'autonomie de l'agent.

Partie III : L'organisation

8. Tous les workers, pas seulement les développeurs

La frontière entre ceux qui codent et ceux qui ne codent pas est en train de disparaître.

La transformation ne concerne pas seulement les développeurs. C’est même l’une des surprises les plus importantes. Un PM, un CSM, un PO ou un designer peuvent utiliser des agents plusieurs fois par jour, gagner du temps, produire un travail plus qualitatif et se concentrer sur des tâches plus complexes. Ils peuvent aussi aller plus loin : créer leurs propres agents, formuler des issues plus précises, assigner des changements à Copilot, valider fonctionnellement des PRs.

Ce n’est pas anecdotique. Quand un PM crée un agent Product Design, il encode une méthode de travail : comment formuler les problèmes design, ce qu’un UX designer doit comprendre, ce qu’il faut éviter, comment rester centré sur le pourquoi et l’impact attendu. La PR #993 sur l’agent **pd-feature-writer** montre exactement cela : un agent qui écrit en Globish, évite les détails d’implémentation, privilégie l’intention design et produit des critères d’acceptation vérifiables par un designer ou un PM.

Quand une CSM formule un changement CMS et l’assigne à Copilot, elle ne contourne pas l’équipe de développement. Elle transforme directement un irritant observé sur le terrain en changement concret, sans attendre qu’un développeur le priorise manuellement. L’issue #986 en est un bon exemple : dans le CMS, les sections repliées affichaient des labels génériques comme **Section 1** ou **Section 2**. Pour un content manager, cela signifiait ouvrir chaque section pour comprendre son contenu. La CSM a créé l’issue, Copilot a ouvert la PR #987, puis elle a validé fonctionnellement que les titres des sections s’affichaient comme attendu.

Ce type de changement de quality of life est essentiel. Ce sont souvent les irritants que les utilisateurs internes ressentent le plus, mais que les équipes repoussent le plus facilement. Ils sont trop petits pour gagner une bataille de priorisation, mais trop utiles pour être ignorés. L’agent change cette économie. Une amélioration CMS comme un color picker avec preview couleur (#937/#985) peut être traitée en moins d’une heure. Un bug mobile visible utilisateur, comme un footer qui ne scrollait pas correctement (#926/#927), peut être assigné à Copilot et valide dans le même flux.

La clé est que cette démocratisation reste gouvernée. Les PMs et CSM ne prennent pas des changements en autonomie hors du flux d’équipe. Les bugs et petites features sont discutés pendant le standup. L’équipe décide collectivement qui s’occupe de quoi : développeur, PM, CSM, agent autonome, ou combinaison des deux.

La règle empirique est simple : si le changement est trivial, à faible impact, et qu’un développeur apporte peu de valeur ajoutée à l’exécution, un PM ou un CSM peut le gérer avec un agent si l’équipe est d’accord. Exemple concret : une CSM rapporte un bug au standup. Le tech lead propose qu’elle le prenne avec Copilot et s’engage à reviewer la PR. L’intention est visible, le mode de

delivery est explicite, et la responsabilité technique reste couverte.

Il n'existe pas encore de liste formelle de sujets interdits. La règle actuelle est prudente : dès qu'un bug ou une feature présente une technicité significative, un développeur le prend en charge pour l'instant. Cela inclut typiquement l'infrastructure, la sécurité, les changements de data model, les migrations, l'authentification, les sujets de performance complexes ou les changements architecturaux. Les bugs de production ne sont pas exclus par principe : un bug visible utilisateur peut être corrigé avec l'aide d'un CSM si le changement reste localisé, faible risque, et que la review technique est assurée.

Cette séparation des responsabilités rend le modèle sain. La validation fonctionnelle reste chez la personne qui porte le besoin : PM, CSM, PO ou designer. La validation technique reste chez le développeur ou le code owner via la PR, les tests, la CI/CD et la review des fichiers critiques. Dans les exemples observés, cette validation technique a majoritairement été faite par le tech lead, code owner de la stack. Mais elle aurait pu être faite par n'importe quel développeur compétent sur la zone.

Ce changement demande des préalables concrets : un environnement de contribution simple pour les non-développeurs, une formation minimale à l'IA, à l'IDE, aux issues, aux Pull Requests et aux validations, puis l'acceptation que des outils réservés aux devs deviennent des outils de travail produit. La confiance vient du fait que l'équipe partage les mêmes agents, les mêmes garde-rails et le même processus de delivery. Le PM ou le CSM n'agit pas avec un outil inconnu, dans un coin, hors du flux ; il utilise les mêmes mécanismes que l'équipe.

L'outil exact peut changer. Le PM peut utiliser VS Code aujourd'hui, Kiro demain, ou un autre environnement plus tard. Ce qui doit rester commun, c'est l'accès au repository, à GitHub, aux instructions, aux agents, aux environnements de PR et aux critères de validation. Les non-développeurs doivent apprendre à tester une fonctionnalité sur un environnement de PR, lire une Pull Request au niveau nécessaire, comprendre le lien avec la user story, vérifier les acceptance criteria et commenter clairement le résultat. Pour faciliter cela, chaque PR devrait exposer les informations utiles : lien vers l'environnement de PR, mode de test, screenshots si nécessaire, risques connus et critères d'acceptation couverts.

Le flux commun n'est donc pas seulement le flux de Pull Requests. C'est le flux complet : de l'idée à la story, de la story au code, du code à la validation, puis de la validation à la production.

La démocratisation ne dilue donc pas la responsabilité. Elle la distribue explicitement entre ceux qui portent l'intention et ceux qui garantissent la qualité technique.

Avant / après

Avant	Après
Les PM/CSM demandent et attendent qu'un dev priorise.	Les PM/CSM peuvent prendre des changements faibles risques avec agent et accord d'équipe.
Les irritants QoL restent en bas du backlog.	Les irritants QoL peuvent être traités rapidement dans le flux normal.
La validation est implicite et concentrée chez les devs.	Validation fonctionnelle et validation technique sont séparées explicitement.

Actions concrètes

1. Identifier les changements QoL faibles risques que PM, CSM ou PO pourraient porter eux-mêmes avec agent.
2. Définir une règle de standup : aucun changement non-dev hors visibilité équipe.
3. Formaliser la matrice de responsabilité et la rendre visible à toute l'équipe dès le début du pilot.

9. La friction comme ennemi principal

Quand l'IA accélère, tout ce qui frottait devient un mur.

La plupart des organisations cherchent d'abord le meilleur modèle, le meilleur prompt ou le meilleur outil. C'est compréhensible, mais incomplet. À partir d'un certain niveau de maturité, la performance ne dépend plus principalement du modèle. Elle dépend de la friction du système autour du modèle.

L'onboarding de l'agent est lui aussi une friction majeure. Il mérite un traitement à part, car il ne s'agit pas d'un simple problème opérationnel : c'est la condition qui permet à l'agent de travailler comme un membre utile de l'équipe. Le chapitre précédent l'a détaillé ; ce chapitre se concentre sur les autres frictions qui apparaissent une fois l'agent en capacité de produire.

La première friction est la capacité de la pipeline. Historiquement, pouvoir exécuter deux ou trois builds en parallèle suffisait. Ce n'était pas un problème parce que le rythme de production humaine était compatible avec cette capacité. Quand les agents arrivent, cette hypothèse s'effondre. Le build time lui-même peut rester raisonnable, par exemple dix à vingt minutes, mais le nombre de changements simultanés augmente. Ce qui bloque n'est plus seulement la durée d'un build, mais le débit global du système.

Dans un cas concret, l'enjeu n'a pas été de réduire massivement le temps de build. Le gain a été de passer d'environ trois builds simultanés à environ vingt builds simultanés, avec plus de ressources allouées aux builds et aux environnements pour éviter les erreurs sporadiques de mémoire. Le tout sans explosion des coûts, grâce au passage d'une infrastructure fixe à une infrastructure préemptible. La bonne métrique n'était donc pas seulement "combien de minutes dure un build ?", mais "combien de changements l'organisation peut-elle valider en parallèle sans perdre le momentum ?".

La CI/CD doit aussi comprendre l'impact réel d'un merge. Tous les changements ne doivent pas déclencher la même chaîne de publication. Un changement de documentation peut publier automatiquement un wiki ou une documentation statique sans générer une nouvelle version applicative. À l'inverse, un changement de dépendances, de code applicatif, de schéma ou d'infrastructure peut nécessiter une nouvelle version du code, des tests complets et un chemin de release plus strict. Cette classification évite de gaspiller de la capacité CI/CD sur des changements sans impact applicatif, tout en garantissant que les changements qui affectent réellement le produit passent par le bon niveau de validation.

La migration d'infrastructure peut aussi créer de nouvelles frictions. Dans ce cas, certains steps sont devenus flaky après une migration vers Azure, alors qu'ils fonctionnaient correctement sur AWS. La migration était nécessaire dans le contexte du partenariat avec Microsoft, mais elle a montré une vérité simple : l'IA ne tolère pas bien les environnements instables. Un développeur humain peut contourner, attendre, relancer, improviser. Un agent a besoin d'un environnement fiable, reproductible et documenté.

Les environnements de PR deviennent alors critiques. Si l'on ne déploie pas systématiquement un environnement de PR, on perd du temps à le déployer manuellement ou à relancer la pipeline après le build. Dix minutes peuvent sembler peu. Mais quand l'agent, le PM, le QA ou le PO attendent pour valider, ces dix minutes cassent le momentum. La vitesse agentique rend coûteuses des frictions qui semblaient auparavant acceptables.

La deuxième friction est la review fatigue. Elle existait déjà avant les agents. Quand une équipe plus grande arrivait en fin de sprint, beaucoup de développeurs committaient, poussaient et rendaient leurs changements visibles tardivement. Le tech lead devait alors revoir toutes les Pull Requests dans un pic de charge. Avec les agents, la forme change : le flux devient plus continu, mais il reste fatigant. Prévoir une heure par jour pour la code review peut suffire dans un régime normal. Mais lorsqu'un changement majeur arrive, il faut parfois bloquer des journées entières, en plus des meetings et des deliveries.

La fatigue de review n'est pas seulement une question de temps. C'est une question de charge mentale. Voir la liste des actions en attente et les PRs qui s'accumulent donne le sentiment d'une vague qui arrive. On sait que chaque PR peut contenir quelque chose d'important. On sait aussi qu'il est impossible de tout revoir avec la même intensité. Cette tension crée de la fatigue, puis des erreurs.

La troisième friction est la QA comme point de validation tardif. Dans le flux traditionnel, les QA et PM/PO validaient souvent le code une fois merge. Chaque correction devenait donc plus longue : il fallait détecter le problème après intégration, revenir vers le développement, corriger, reconstruire, redéployer, puis revalider. La boucle de feedback arrivait trop tard.

Le modèle de sprint accentuait ce problème. Les QA devaient valider 100% des user stories, mais la charge n'était pas régulière. En début de sprint, si peu de nouvelles epics étaient prêtes, la QA pouvait être idle. En fin de sprint ou lors des releases, elle devenait surchargée. Le besoin ressemblait à un QA temps plein, mais pas de manière constante. Cette irrégularité crée de l'attente, de la pression et une qualité de validation moins stable.

L'absence de régression automatique renforçait encore cette friction. Elle n'existait pas parce que l'équipe n'avait pas eu le temps de la créer, et surtout parce qu'elle n'avait pas le temps de la maintenir. C'est un cercle classique : faute d'automatisation, la QA manuelle devient indispensable ; parce que la QA manuelle consomme tout le temps disponible, l'automatisation n'avance pas.

La quatrième friction est l'agenda. Le travail avec agents demande du temps de concentration, de supervision et de review. Or les réunions cassent ce flow. La réduction la plus efficace n'est pas toujours de supprimer des cérémonies officiellement, mais de vider l'agenda des réunions où l'on n'est pas indispensable. Lire les minutes au besoin, intervenir uniquement quand l'avis est nécessaire,

réduire les meetings clients non essentiels, rendre les refinements plus focalisés : tout cela redonne du temps de travail réel.

Une équipe réduite aide aussi. Moins de personnes à coordonner signifie moins de réunions, moins de préparation, moins de synchronisation implicite. L'équipe gagne en autonomie, et le tech lead récupère du temps pour les activités qui deviennent critiques dans un SDLC agentique : review, architecture, calibration des agents, documentation et suppression des frictions.

Friction : symptômes et corrections

Symptôme	Cause probable	Correction
PRs en attente malgré des builds rapides	Capacité parallèle insuffisante	Augmenter le nombre de builds simultanés et dimensionner les ressources.
Momentum perdu après le build	Environnements de PR non déployés systématiquement	Déployer automatiquement les environnements de PR.
Erreurs aléatoires de pipeline	Environnement flaky ou ressources insuffisantes	Stabiliser les steps, augmenter les ressources, rendre les tests reproductibles.
Review fatigue	Flux continu ou pics de PRs trop importants	Dédier du temps de review, prioriser par risque, utiliser tests et agents reviewers.
QA trop tardive	Validation uniquement après merge	Déplacer la validation fonctionnelle avant merge via environnements de PR.
QA idle puis surchargée	Charge concentrée en fin de sprint ou release	Lisser le flux, réduire le batch size, automatiser la régression.
Pas de régression automatique	Pas le temps de créer ni maintenir les tests	Investir dans un socle E2E stable et limité aux parcours critiques.
Flow cassé	Trop de réunions ou présence non indispensable	Réduire l'agenda, lire les minutes, rendre les refinements plus focalisés.

Avant / après

Avant	Après
La CI/CD valide quelques changements humains.	La CI/CD doit absorber un flux parallèle humain + agents.

Avant	Après
La review fatigue arrive en fin de sprint.	La review fatigue devient continue si le flux n'est pas gouverné.
QA/PM/PO valident après merge.	La validation fonctionnelle se déplace avant merge quand un environnement de PR existe.

Actions concrètes

1. Mesurer le débit CI/CD, pas seulement le temps d'un build individuel.
2. Automatiser les environnements de PR pour conserver le momentum de validation.
3. Réduire les réunions ou la présence du tech lead n'est pas indispensable.

10. Repenser l'organisation sans juste renommer les rôles

Renommer un Scrum Master en AI Orchestrator ne transforme rien.

L'IA rend intenables certaines limites déjà présentes dans les organisations Agile. Le sprint fonctionne bien quand l'équipe maîtrise relativement son sujet, sa capacité et ses coûts. On définit une équipe, on estime un volume de travail, on espère que le système se comporte comme une horloge. Mais quand l'exécution devient beaucoup plus rapide, plus parallèle et plus organique, ce modèle montre ses limites.

Les cérémonies n'ont pas besoin de disparaître. Dans l'expérience de l'équipe, elles sont surtout devenues plus efficaces. Le standup, en particulier, est devenu plus organique : on y gère les nouveaux tickets au jour le jour, avec les nouvelles priorités, dans une logique proche du Kanban. Ce mode hybride — Scrum pour la structure, Kanban pour le flux continu — porte souvent le nom de Scrumban. Il n'est plus seulement un tour de table de statut, mais un point de régulation du flux.

Ce changement est subtil mais important. Quand un bug arrive, l'équipe ne le pousse pas automatiquement dans un sprint futur. Elle décide qui doit le prendre maintenant : un dev, un PM, un CSM, Copilot, un agent local, ou une combinaison. Quand une priorité change, elle peut être intégrée plus rapidement. Quand un changement est trivial, faible risque et clair, il peut être traité sans attendre une nouvelle cérémonie de planification.

Les raffinements deviennent eux aussi plus focalisés. On passe moins de temps à coordonner pour coordonner. On passe plus de temps à clarifier les besoins vraiment incertains, les impacts, les risques, les décisions de produit et les limites de scope. L'IA accélère l'exécution, mais elle ne remplace pas la réflexion produit. Elle rend simplement plus visible le fait que la réflexion produit est souvent le vrai goulot.

La taille d'équipe change également la dynamique. Dans mon projet test, nous livrons aujourd'hui autant qu'à l'époque où l'équipe comptait environ douze personnes, mais avec une équipe beaucoup plus réduite. Le gain ne vient pas seulement de l'IA qui produit du code plus vite ; il vient aussi de la baisse massive du coût de coordination. Moins de personnes signifie moins de communication indirecte, moins de dépendances internes et moins de synchronisation. Les décisions circulent plus vite, les problèmes deviennent visibles plus tôt, les arbitrages se font avec moins de friction.

La comparaison avant/après n'est pas seulement une réduction de capacité. C'est aussi une recomposition. Le rôle de CSM n'existait pas dans l'équipe initiale. Il a été ajouté délibérément parce qu'il était important pour notre projet : rapprocher la voix client du flux de delivery. L'équipe n'a donc pas simplement rétréci ; elle a changé de forme.

Rôle	Avant	Après
Développeurs	3 Frontend (FE) + 3 Backend (BE)	2.5 (full-stack)
QA	1 (100%)	0.2 (1j/semaine, expert transverse)
DevOps	1 (embarqué)	0.3 (support externe)
PM + PO	2 personnes distinctes	1 (rôles fusionnés)
UX Designer (UXD)	1	0.5
CSM	— (n'existait pas)	1 (rôle ajouté)
Total ETP (Équivalent Temps Plein)	~12	~4.5

Ce tableau doit être lu avec précaution. Il décrit un contexte de prototypage et d'expérimentation, pas un modèle universel. Il ne dit pas qu'il faut supprimer des rôles. Il dit que la fréquence et la criticité du besoin doivent guider la composition de l'équipe plutôt que les conventions héritées.

Un dernier point sur la composition développeur : les 2.5 ETP incluent un développeur junior, encadré par deux seniors. Ce n'est pas un détail. Dans un setup agentique bien construit — documentation explicite, guardrails versionnés, review structurée, environnement reproductible — un junior peut contribuer avec confiance dès le départ. Les conditions qui rendent l'agent fiable sont les mêmes qui rendent l'onboarding d'un junior plus sûr : le contexte est écrit, les règles sont claires, et les seniors voient tout passer par les PRs.

Mais cette compression à un coût. Une petite équipe est moins résiliente. Si plusieurs personnes sont en vacances ou malades, le pourcentage de capacité perdu est beaucoup plus important. L'équipe livre moins, voire ne livre plus sur certains sujets. Elle dépend davantage de fonctions de support ou d'expertises externes qu'elle doit aller chercher ailleurs au besoin. Comme dans toute petite équipe, un besoin urgent peut absorber une part disproportionnée de la capacité disponible.

Il faut donc éviter de présenter la réduction d'équipe comme une recette universelle ou un objectif RH. L'objectif n'est pas de faire le même travail avec moins de personnes par dogme. L'objectif est de réduire la coordination inutile et de concentrer les compétences là où elles apportent vraiment de la valeur. Les fonctions de support restent nécessaires. L'expertise reste nécessaire. Mais elle peut être mobilisée ponctuellement plutôt que portée en permanence par l'équipe de delivery.

Le modèle cible ressemble davantage à une équipe réduite, autonome, fortement outillée, entourée d'experts accessibles. La plupart des rôles deviennent des skills : QA, DevOps, sécurité, UX, architecture, legal, accessibilité. Tout le monde ne devient pas expert en tout ; l'équipe doit identifier les compétences dont elle a besoin en continu, celles qu'elle peut apprendre, et celles qu'elle doit aller chercher ponctuellement.

Dans le setup de prototypage décrit ici, la QA est surtout une ressource d'expertise externe et transverse. Elle intervient pour renforcer la validation, challenger les critères d'acceptation, vérifier les parcours critiques ou accompagner les releases. Cela ne doit pas être lu comme une recommandation de supprimer la QA. Dans un produit avec plus d'utilisateurs, plus de SLA, plus de risques business ou réglementaires, un focus QA plus fort devient probablement indispensable.

L'agent QA a tout de même un rôle utile dans le système. Il peut comparer la user story à la Pull Request, tester la delivery, vérifier les acceptance criteria et expliciter ce qui est valide, invalide ou non prouvé. Il ne remplace pas la responsabilité qualité d'une vraie compétence QA ; il rend la validation plus précoce, plus explicite et plus répétable.

L'exemple du design est utile parce qu'il force l'humilité. Une IA peut produire un joli design. Cela ne veut pas dire que le design résout correctement le problème utilisateur, respecte les critères d'accessibilité, s'intègre au système existant, ou prend les bons arbitrages d'interaction. Quelqu'un sans affinité UXD peut obtenir une proposition visuelle acceptable avec une IA, mais ne pas avoir le skill nécessaire pour juger si la solution design est vraiment bonne. À l'inverse, un UX designer peut lui aussi développer des compétences de développement et utiliser les agents pour aller plus loin dans la formalisation ou l'exécution.

La vraie question organisationnelle n'est donc pas "avons-nous encore besoin de tel rôle ?". C'est : "de quelle compétence avons-nous besoin, à quelle fréquence, avec quel niveau de risque ?". Si l'équipe a besoin d'une compétence à 80%, 100% ou 300% du temps, cette compétence doit probablement être dans l'équipe. Si elle en a besoin à 10%, elle peut venir d'un expert externe, d'une équipe tierce, ou d'un membre de l'équipe qui se forme pour couvrir ce manque. Si le besoin est faible mais récurrent, deux options sont saines : former quelqu'un dans l'équipe, ou établir un accord clair avec une équipe transverse capable d'absorber les requêtes et de s'organiser.

Ce raisonnement n'a rien de magique. C'est de la gestion d'équipe classique, rendue plus visible par l'IA. Une équipe resserrée fonctionne si elle dispose d'un accès presque illimité aux expertises externes dont elle a besoin : sécurité, legal, UX, performance, infrastructure, architecture. Elle doit pouvoir dire ce qui lui manque, apprendre quand c'est raisonnable, ajouter un membre quand le besoin devient permanent, ou "appeler un ami" quand le besoin reste ponctuel.

Ce modèle permet aussi à l'équipe de grandir. Elle n'est pas enfermée dans une composition rigide. Elle observe ses manques, développe de nouveaux skills, et renforce son staff uniquement quand la fréquence et la criticité du besoin le justifient.

L'Agile sous agents n'est donc pas moins discipliné. Il est moins cérémoniel. Il privilégie le flux, la clarté, la réversibilité, la validation rapide et la responsabilité explicite.

Avant / après

Avant	Après
Le standup sert surtout à partager le statut.	Le standup devient un point de régulation du flux et des priorités.
Le sprint absorbe les changements par batch.	Les tickets et priorités sont gérés plus organiquement, en mode Scrumban.
Les rôles sont vus comme des postes fixes.	Les rôles sont analysés comme des skills nécessaires à différentes fréquences.
Une équipe large livre avec plus de couverture mais beaucoup de coordination.	Une équipe réduite peut livrer autant avec moins de coordination, si elle a accès aux expertises externes.

Actions concrètes

1. Transformer le standup en point de décision sur le flux, pas seulement en reporting.
 2. Garder les cérémonies utiles, mais les rendre plus courtes, plus ciblées et plus décisionnelles.
 3. Assumer les limites d'une petite équipe : support externe, backup, gestion des absences, risques d'urgence.
 4. Identifier les skills nécessaires en continu, les skills à apprendre, et les expertises à solliciter ponctuellement.
-

11. Le management comme premier vecteur du changement

Une équipe ne deviendra pas agentique si son management reste spectateur.

Le rôle du management n'est pas seulement d'acheter des licences ou de déclarer que l'IA est stratégique. Son rôle est de créer les conditions concrètes dans lesquelles l'équipe peut apprendre à travailler avec des agents : temps, confiance, droit à l'erreur, accès aux experts, environnement technique, objectifs clairs et absence de pression prématurée sur le résultat.

Les hackathons ont fortement accéléré l'adoption parce qu'ils étaient préparés. Tout le monde devait comprendre les outils, l'objectif de l'exercice et surtout ce qui n'était pas attendu. L'objectif n'était pas de livrer un produit parfait en deux jours, mais de vivre l'agentique development dans un cadre protégé.

Le pairworking avec des sachants est crucial. Travailler avec des experts, comme les équipes ISE de Microsoft, permet de faire un bond de géant. Non pas parce que les experts font le travail à la place de l'équipe, mais parce qu'ils accélèrent la boucle d'apprentissage. Ils aident à diagnostiquer ce qui bloque, à distinguer un problème de prompt d'un problème d'environnement, à choisir le bon niveau de délégation, à récupérer plus vite quand l'agent part dans la mauvaise direction.

Le management doit surtout faire confiance sans exiger un résultat immédiat. Cette combinaison est rare et précieuse : confiance forte, pression faible. On n'est pas là pour prouver que tout marche déjà, mais pour échouer suffisamment vite et clairement afin de comprendre ce qui bloque.

L'échec n'est pas une anomalie dans l'adoption des agents. C'est une étape du processus. Les agents vont mal interpréter des instructions, ignorer des règles, manquer du contexte ou affirmer avoir fini alors que ce n'est pas vrai. Ce n'est pas automatiquement parce que les agents sont nuls, ni parce que les humains sont nuls. C'est souvent parce que l'équipe, l'environnement, les instructions et les outils ne sont pas encore prêts.

C'est le changement culturel à protéger. Dans beaucoup d'organisations, l'échec est toléré en théorie mais sanctionné en pratique. Avec les agents, cette posture bloque l'apprentissage. Le management doit donc expliciter l'espace d'expérimentation : tester, échouer, comprendre, corriger, recommencer. Les retours d'expérience doivent pointer ce qui manque dans la documentation, l'environnement, les outils, les garde-rails ou les skills.

La plus grosse friction humaine observée ne venait pourtant pas d'une peur abstraite de l'IA. Elle venait du manque d'alignement entre les objectifs de chacun. Certains voulaient absolument faire évoluer le produit. D'autres venaient avec la mission de mettre en place des agents autonomes. D'autres voulaient surtout apprendre à travailler avec des agents. D'autres encore voulaient transformer la méthodologie de travail de l'équipe dans son ensemble.

Tous ces objectifs étaient légitimes. Mais ils n'étaient pas équivalents, et ils ne

conduisaient pas aux mêmes arbitrages. Si l'objectif principal est de livrer le produit, on optimise pour la feature. Si l'objectif principal est de construire des agents autonomes, on accepte de ralentir la delivery pour investir dans l'infrastructure agentique. Si l'objectif est l'apprentissage, on accepte l'échec et la répétition. Si l'objectif est la transformation méthodologique, on touche aux rituels, aux rôles et aux responsabilités. Mélanger ces objectifs sans les expliciter produit de la friction.

Ce manque d'alignement à génère des mésententes, de la frustration et parfois des tensions inutiles. C'est probablement un échec de communication des objectifs, ou d'alignement du management envers les équipes. Et cet échec peut devenir délétère : chacun croit travailler pour la bonne chose, mais personne ne travaille exactement pour la même chose.

La leçon est importante. Avant de lancer une transformation agentique, le management doit clarifier l'objectif dominant de chaque phase. Est-on là pour livrer une feature ? Pour apprendre ? Pour construire l'infrastructure d'agents ? Pour changer le mode de travail ? Pour prouver un ROI ? Ces objectifs peuvent coexister, mais ils doivent être ordonnés. Sinon l'équipe ne sait pas comment arbitrer quand ils entrent en conflit.

La sécurité psychologique ne se décrète pas en réunion générale. Elle se construit dans les conditions concrètes du travail : hackathons bien préparés, experts disponibles, objectifs explicites, absence de pression prématurée. C'est ce qui donne aux équipes la permission d'apprendre par la pratique.

Le manager doit enfin se former lui-même. Il ne peut pas piloter cette transformation uniquement à travers des slides. Il doit utiliser les agents, sentir les frictions, comprendre pourquoi une tâche simple échoue, voir ce que change un bon contexte, expérimenter la fatigue de supervision, comprendre le moment où l'agent devient vraiment utile. Sans cette expérience, il risque de poser les mauvaises questions et de fixer les mauvais objectifs.

Avant / après

Avant	Après
Le management demande l'adoption depuis l'extérieur.	Le management crée les conditions concrètes d'apprentissage.
Le hackathon cherche une demo réussie.	Le hackathon cherche des échecs utiles et des apprentissages transférables.
L'échec d'un agent est vu comme une preuve d'incompétence.	L'échec est analysé comme un signal : contexte, outil, environnement, méthode.
Les objectifs implicites coexistent.	L'objectif dominant de chaque phase est explicite : livrer, apprendre, industrialiser ou transformer.

Actions concrètes

1. Organiser des hackathons agentiques avec préparation, objectifs clairs et absence d'attente de résultat immédiat.
 2. Mettre les équipes en pairworking avec des sachants pour accélérer la boucle d'apprentissage.
 3. Traiter chaque échec agentique comme une source de diagnostic, pas comme une faute.
 4. Aligner explicitement les objectifs avant chaque phase : produit, apprentissage, agents autonomes, méthodologie ou ROI.
-

Partie IV : Gouverner et passer à l'échelle

12. Le coût de la transformation et son ROI

L’IA n’est pas gratuite. Le statu quo non plus.

Le premier coût visible est la licence. Dans un usage intensif, les crédits standards ne suffisent pas toujours. Un ordre de grandeur observé est d’environ 3 000 crédits Copilot par ingénieur et par mois, soit jusqu’à dix fois le package standard. Ce chiffre doit être budgété explicitement. Sinon l’organisation adopte officiellement l’IA tout en limitant concrètement son usage.

Le deuxième coût est le setup. Deux mois est le strict minimum pour un démarrage from scratch — non pas pour avoir un setup parfait à la fin, mais pour commencer à comprendre ce qui fonctionne et ce qui ne fonctionne pas, et obtenir quelque chose de fonctionnel et répliquable. Ce délai n’est pas seulement du temps technique. C’est aussi le temps nécessaire pour que l’équipe adopte et dompte les agents et le workflow. Sans cette durée minimale, la transformation ne peut pas avoir lieu.

Ce point est important à comprendre : le temps de setup est le temps d’adoption. L’équipe a besoin de ce temps pour identifier ce qui manque, analyser, échouer, puis trouver ou construire les agents, skills, prompts, tools et documentation qui font fonctionner l’expérience. Couper cette durée par pression managériale, c’est empêcher l’équipe d’apprendre ce dont elle a réellement besoin.

Avec un framework, des outils et des agents réutilisables déjà disponibles — et l’expertise accumulée pour les utiliser — cette durée peut être divisée par deux environ. Pour un contexte très différent, avec une stack exotique, un domaine fortement réglementé ou une architecture legacy, il faudra probablement plus de temps.

Le troisième coût est la formation. Une équipe doit apprendre à utiliser les agents, mais aussi à changer sa méthode de travail : découper les tâches, donner le contexte, reviewer autrement, accepter l’échec, maintenir les instructions, ne pas confondre vitesse et qualité. Ce n’est pas une formation outil. C’est une formation au travail avec un nouveau type de collaborateur.

Le quatrième coût est l’infrastructure. La CI/CD, les environnements de PR et les ressources de build doivent suivre. Dans mon projet test, le temps de build individuel n’a pas été radicalement réduit : il reste autour de dix à vingt minutes. Le vrai gain est venu du débit : passer d’environ trois builds simultanés à environ vingt builds simultanés, avec plus de ressources pour éviter les erreurs de mémoire, sans augmenter les coûts grâce au passage d’une infrastructure fixe à une infrastructure préemptible.

Ce point est important pour calculer le ROI. L’objectif n’est pas seulement de faire un build plus vite. L’objectif est d’éviter que des agents, des développeurs, des PMs ou des QA attendent la pipeline. Chaque attente casse le momentum. Chaque environnement de PR non disponible retarde une validation. Chaque step flaky consomme du temps humain. L’infrastructure n’est plus un centre de

coût passif. Elle devient un multiplicateur ou un frein de productivité.

Le coût le plus important reste celui de ne pas adopter. Les concurrents utiliseront ces outils. Les talents voudront travailler avec ces outils. Les équipes qui apprendront à transformer leur connaissance fonctionnelle en contexte exécutable iront plus vite. Elles auront une meilleure documentation, une meilleure CI/CD, des workflows plus explicites, des feedback loops plus courts. Même si l'IA devait décevoir, ces actifs resteraient utiles.

Le ROI ne doit donc pas être mesuré en lignes de code. Il se voit dans la capacité à livrer autant avec une équipe plus réduite, dans la baisse du coût de coordination, dans la vitesse de résolution des irritants, dans la réduction des commentaires de PR sur les changements bien préparés, dans l'implication plus précoce des PM/PO/QA, dans la capacité à traiter des quality of life changes qui restaient auparavant bloqués dans le backlog.

Mais il faut rester honnête : tout le monde ne gagne pas immédiatement. Le début est coûteux. Les agents échouent. Les instructions changent. L'environnement doit être stabilisé. Les objectifs doivent être alignés. Les reviewers fatiguent. La dette d'information devient visible. Le ROI arrive quand l'organisation accepte de payer ce coût initial au lieu de chercher une magie instantanée.

Coût Total de Possession (TCO)

Poste	Ordre de grandeur / effet
Licences IA	Jusqu'à 3 000 crédits Copilot par ingénieur et par mois dans un usage intensif.
Setup agents/workflows	Quelques jours si préparé, plusieurs semaines à deux mois from scratch.
Formation	Environ deux semaines d'adaptation active pour changer les habitudes de travail.
CI/CD	Débit plus important que build time individuel : passage observé d'environ 3 à 20 builds simultanés.
Infrastructure	Passage à des ressources préemptibles pour augmenter la capacité sans augmenter les coûts.
Classification des changements	Distinguer documentation, dépendances, code applicatif, schéma et infrastructure pour déclencher le bon chemin de publication.

Investissement total et ROI

L'expérimentation complète représente environ 3 mois d'investissement concentré :

- Phase 1 (1 mois) : ~4 ETP (tech lead/architect, DevOps, PM/PDA, expert agentic).
- Phases 2-3 (2 mois) : ~7 ETP (équipe pilot complète + support externe).

- **Total** : ~18 ETP-mois (équivalent de 18 personnes mobilisées 1 mois chacune).

Ce n'est pas un coût supplémentaire net : l'essentiel de cet investissement est du temps d'équipe existante redirigé vers l'expérimentation. Le retour se matérialise par la réduction du coût de coordination et la capacité de l'équipe à livrer avec moins de personnes. Sur une équipe initiale de 10-12 personnes, le ROI se situe entre 3 et 6 mois après la fin de l'expérimentation. L'équipe réduite peut ensuite accélérer sur le projet en cours et le modèle peut être répliqué sur d'autres projets.

Critères de continuation

Ne pas juger l'expérimentation sur le résultat du premier mois. Sur mon projet test, si nous avions fait le bilan à un mois, nous aurions tout arrêté. La migration à Azure avait été catastrophique, les agents ne fonctionnaient pas bien, tout allait de travers. Le retour de notre management et de Microsoft a été : l'échec est normal. Maintenant qu'on a quelque chose qui fonctionne suffisamment, on expérimente et on apprend dessus, on fixe au fur et à mesure. Et on y est arrivé — dans la douleur, mais ça s'est fait parce que l'équipe a été moteur et que le top management a fourni un support sans faille.

Les vrais critères de continuation ne sont pas des métriques de productivité au premier mois. Ce sont : la confiance dans le système (l'équipe utilise le setup naturellement), des progrès mensuels visibles (chaque mois apporte des améliorations concrètes), un support management qui survit aux échecs, et un engagement équipe qui reste positif. Le go/no-go n'est pas binaire — c'est une évaluation mensuelle de la trajectoire, pas du résultat instantané.

Risques et mitigations

Risque	Mitigation
Échec au setup initial	L'équipe n'arrive pas à structurer ou récupérer l'information. Si le projet cible est trop complexe, choisir un projet plus petit ou investir plus de temps sur le setup. Mieux vaut repousser l'implémentation sur un gros projet que de l'aborder prématurément. Microsoft a documenté un success story sur .NET avec des centaines de contributeurs (voir Références).
Résistance au changement	Ne pas chercher à convaincre tout le monde d'emblée. Commencer par des early adopters motivés et convaincus. Le scepticisme se radoucit par la multiplication de petits projets réussis, pas par des arguments théoriques.

Risque	Mitigation
Perte de compétence	Le risque existe. Mais la code review fait monter les compétences de programmation : c'est comme lire un livre et poser des questions dessus. Il n'y a pas de perte sèche de connaissance si les équipes travaillent sérieusement. En revanche, certains automatismes seront oubliés — comme lorsqu'on crée un script pour se simplifier la vie et qu'on oublie ce qu'il y avait dedans. C'est aux ingénieurs de garder un minimum d'hygiène de développement.
Turnover et interruptions	Sur mon projet test, le sachant IA a quitté l'entreprise en cours d'expérimentation. Les nouveaux interlocuteurs avaient d'autres compétences et d'autres façons de voir ; on a appris différemment et c'était enrichissant. Vacances longues, hackathons, contraintes sur plusieurs semaines : tout cela impacte la delivery, mais l'apprentissage se fait et on progresse si le setup est versionné et documenté.

Sécurité et conformité

La question de la sécurité revient systématiquement dans les discussions management. Elle est légitime mais ne doit pas devenir un prétexte pour ne pas avancer. Les outils majeurs (GitHub Copilot, Claude Code, Kiro) ne retiennent pas le code soumis pour l'entraînement en mode entreprise — vérifier les conditions contractuelles. Le code généré par un agent est soumis aux mêmes règles que le code écrit par un humain : il passe par une PR, il est reviewé, il appartient à l'entreprise. Le périmètre d'accès des agents est défini explicitement dans les instructions projet versionnées. Et l'agent ne déploie pas en production sans validation humaine — les mêmes contrôles s'appliquent. La sécurité est un sujet de configuration, pas un blocker fondamental.

Coût de l'inaction

Ne pas expérimenter maintenant n'est pas une position neutre. Le coût de l'inaction ne se mesure pas encore en euros — le ROI précis est en cours d'évaluation et le scale vers d'autres projets demande d'analyser comment chacun fonctionne pour adapter les méthodes. Ce qui se mesure déjà, c'est le retard d'apprentissage : il est cumulatif et non-linéaire. Les organisations qui expérimentent aujourd'hui accumulent des mois de savoir-faire sur le setup agentique, les échecs, les patterns qui marchent. Ce savoir ne s'achète pas — il se construit par l'itération. L'écart se creuse chaque mois. L'attractivité se dégrade. Et la transformation subie est toujours plus coûteuse que la transformation choisie.

Avant / après

Avant	Après
Le coût IA est vu comme une licence outil.	Le coût IA inclut licences, setup, formation, infra, documentation de référence et support.
La CI/CD est dimensionnée pour une production humaine.	La CI/CD est dimensionnée pour un flux humain + agents.
Le ROI est cherché dans les lignes de code.	Le ROI est cherché dans le flux, la qualité, la coordination et la vitesse de validation.

Actions concrètes

1. Budgeter les crédits IA sur l'usage réel, pas sur le package standard.
2. Chiffrer le coût d'une pipeline capable d'absorber le flux parallèle.
3. Mesurer le coût du statu quo : lenteur, attrition, dette d'information, opportunités manquées.

13. Mesurer ce qui compte

Les lignes de code ne mesurent plus la productivité. Elles mesurent parfois le bruit.

Les métriques traditionnelles deviennent insuffisantes, voire dangereuses. Les story points perdent une partie de leur sens quand l'exécution varie fortement selon la qualité du contexte, la maturité de l'agent, la réversibilité de la tâche et l'état de la CI/CD. Les lignes de code deviennent absurdes quand une journée peut produire beaucoup de documentation, de boilerplate, de framework ou de code généré.

La bonne unité de mesure devient le flux et la friction.

Le temps de fermeture d'un bug est une métrique simple et puissante : du moment où il est ouvert jusqu'au fix valide en test, puis idéalement jusqu'à la production via les métriques DORA. Le volume de Pull Requests doit être suivi avec nuance : PRs ouvertes pour mesurer le rythme de production, PRs mergées pour mesurer la delivery réelle, PRs fermées non mergées pour mesurer l'expérimentation. Une hausse des PRs fermées non mergées n'est pas forcément un échec. Elle peut indiquer que l'équipe explore plus vite, jette plus vite, apprend plus vite.

La capacité du système de validation doit aussi devenir visible : nombre de builds simultanés, temps d'attente avant build, temps de disponibilité des environnements de PR, taux de flaky tests, temps entre PR ouverte et validation fonctionnelle, temps entre validation fonctionnelle et merge. Dans un SDLC agentique, le goulot se déplace souvent vers ces zones.

L'adoption doit elle aussi être observable. Combien de personnes utilisent les agents ? Quels agents ? Quels skills ? Quels tools ? À quelle fréquence ? Quels sont les outliers : ceux qui utilisent beaucoup, ceux qui n'utilisent jamais, ceux qui utilisent mal ? Sans telemetry minimale, l'organisation ne sait pas si elle transforme le travail ou si elle accumule des initiatives individuelles invisibles.

La qualité humaine du changement doit enfin être mesurée. Le Net Promoter Score (NPS) de la méthode de travail peut sembler moins technique, mais il est utile. Les équipes se sentent-elles plus autonomes ? Les PM/PO/QA se sentent-ils plus impliqués ? Les développeurs font-ils confiance au flux ? Le management a-t-il clarifié les objectifs ? La satisfaction n'est pas un bonus : elle conditionne l'adoption durable.

Dashboard proposé

Dimension	Métrique	Pourquoi
Flux	PRs ouvertes, mergées, fermées non mergées	Distinguer production, delivery et expérimentation.

Dimension	Métrique	Pourquoi
Cycle time	Bug ouvert -> fix valide -> prod	Mesurer la résolution réelle.
CI/CD	Builds simultanés, attente avant build, flaky rate	Détecter le goulot technique.
Validation	Temps PR ouverte -> env PR -> validation fonctionnelle -> merge	Mesurer la boucle complète, pas seulement le build.
Qualité	Rollbacks, incidents, commentaires critiques, tests cassants	Vérifier que la vitesse ne dégrade pas le produit.
Adoption	Usage agents/skills/tools par personne	Identifier adoption réelle et outliers.
Gouvernance	Drift skills, prompts obsolètes, tools non utilisés	Maintenir l'infrastructure IA sans imposer une méthode unique.
Satisfaction	NPS méthode de travail	Mesurer l'adhésion humaine.

Métriques à éviter

Métrique	Pourquoi elle trompe
Lignes de code	Mélange logique métier, doc, boilerplate, framework et code généré.
Nombre brut de PRs	Encourage le bruit si on ne distingue pas merge, abandon et expérimentation.
Story points seuls	Capture mal la variabilité liée au contexte, aux agents et à la réversibilité.
Taux d'utilisation IA seul	Utiliser beaucoup un agent ne signifie pas produire de la valeur.

Actions concrètes

1. Suivre PRs ouvertes, mergées et fermées non mergées comme trois signaux distincts.
2. Mesurer la boucle complète de validation, pas seulement le temps de build.
3. Instrumenter l'usage des agents, skills et tools pour détecter adoption, dérive et obsolescence.

14. Feuille de route : Discovery, Pilot, Scale, Govern

La première action n'est pas une stratégie. C'est un setup sur une vraie machine.

La transformation commence rarement par un grand plan. Elle commence quand quelqu'un s'assoit avec une personne de l'équipe et configure son environnement sur son vrai projet. Pas une démo générique. Pas une présentation. Son laptop, son repository, ses contraintes corporate, son firewall, son VDI, ses tests, ses douleurs.

L'environnement peut saboter l'adoption avant même qu'elle commence. Firewall, VDI, accès réseau, droits Git, lenteur des installs, secrets, environnements locaux impossibles : tout cela doit être résolu en amont. Ce ne sont pas des excuses. Ce sont des frictions réelles.

Une feuille de route pragmatique peut suivre six phases.

Requirement 0 : l'organisation est-elle prête ? Microsoft appelle cela le "Requirement 0". C'est le prérequis évalué par l'équipe de support externe avant de décider si oui ou non ils investissent 3-4 ETP pour accompagner le changement. L'expérimentation agentique exige un trust à 200% du management (pas un accord tiède, un mandat clair qui survit au premier échec), des acteurs owners de leur sujet à 200%, une motivation à toute épreuve, l'ownership de bout en bout pour l'équipe pilot, et l'absence de goulots d'étranglement organisationnels. S'il faut passer par un process, demander des autorisations, attendre des semaines de validation pour chaque décision technique — l'organisation n'est pas prête.

Échouer au setup n'est pas un échec. C'est un apprentissage. Cela ne signifie pas qu'il faut arrêter le changement, mais identifier les blockers et débloquer la situation. Les deux sponsors jouent des rôles complémentaires : le sponsor externe (Microsoft/AWS) challenge l'équipe, accepte l'échec comme normal, et peut recommander de repousser la suite si le terrain n'est pas prêt. Le sponsor interne (management) escalade les problèmes organisationnels et débloque les process internes que l'équipe ne peut pas résoudre seule.

Phase 0 : Alignement management (*1 à 2 semaines*). Avant le pilot, clarifier l'objectif dominant : apprendre, construire des agents autonomes, transformer la méthode, prouver un ROI ou livrer plus vite. Livrer plus vite ne doit pas être l'objectif initial. Si le management exige une accélération immédiate, l'équipe cherchera à prouver un résultat avant d'avoir construit le setup, compris les échecs et stabilisé les agents. La vitesse vient plus tard, comme conséquence d'un système qui fonctionne.

Phase 1 : Préparer le terrain, pas tout l'agentique (*2 à 4 semaines selon maturité existante, ~4 ETP : 1 tech lead/architect, 1 DevOps, 1 PM/PDA pour audit et préparation repo, 1 expert agentique pour accompagner l'adoption*). Identifier les frictions du SDLC actuel et vérifier que le terrain non-IA est prêt. Où l'information est-elle cachée ? Où la CI/CD bloque-t-elle ? Quels environ-

nements ne se lancent pas ? Quels documents sont obsolètes ? Quels repositories doivent être rapprochés ? Quels accès manquent ? L’objectif n’est pas de concevoir tous les agents à l’avance, mais d’éviter que l’expérimentation échoue sur des problèmes que n’importe quel newcomer rencontrerait déjà.

Phase 2 : Onboarder l’agent (*2 à 4 semaines, puis amélioration continue*). Construire progressivement les instructions projet, les premiers agents développeur, PM/story writer, review et QA, les tools déterministes, les accès utiles, les guardrails et la définition du done agentique. Cette phase doit rester organique : les équipes ne savent pas encore exactement quels agents, skills ou workflows seront utiles. En revanche, elles ne doivent pas partir d’une page blanche. Des bases de connaissances et des exemples peuvent être mis à disposition comme matériau d’inspiration, notamment via un framework **ai-artifacts** référencé en fin de document. Les équipes peuvent y piocher, adapter, supprimer et versionner ce qui leur sert réellement. Former aussi les non-développeurs à lire une Pull Request, accéder à l’environnement de PR, tester la fonctionnalité, valider ou invalider les acceptance criteria et commenter clairement le résultat.

Phase 3 : Pilot avec 1 à 2 équipes (*6 à 8 semaines, ~7 ETP par équipe pilot*). Choisir une équipe restreinte d’early adopters qui vont itérer rapidement pour apprendre : 2 développeurs, 1-2 fonctionnels (QA/PDA/PM), quelques fonctions de support observatrices (architecture, sécurité, DevOps) qui suivent le pilot pour ensuite évangéliser le reste de l’organisation. L’équipe doit inclure un tech lead impliqué, un PM/PO disponible, un QA ou validateur fonctionnel. Microsoft et AWS proposent d’embarquer 2 à 4 ingénieurs pendant les premières semaines d’adoption — ce coût d’accompagnement fait partie du pricing d’adoption chez les fournisseurs cloud. Commencer par petits bugs localisés, quality of life changes internes, documentation, tests et refactorings faibles risques. Éviter au début l’architecture structurante, la sécurité critique, les migrations de données, l’authentification, la performance complexe et les changements irréversibles.

Le vocabulaire peut varier selon les organisations. AWS AI-DLC parle d’Inception, Construction et Operations, avec des cycles plus courts appelés “bolts” plutôt que sprints. Il n’est pas nécessaire de reprendre cette terminologie. Ce qui compte est le principe sous-jacent : des cycles plus courts, plus collaboratifs, où l’IA conserve le contexte et où les humains valident les décisions structurantes.

Phase 4 : Adapter les rituels (*pendant le pilot*). Garder les cérémonies utiles, mais les adapter au flux : standup comme point de régulation et de délégation, raffinement plus court et centré sur impact/risque/réversibilité, démos fréquentes sur environnements de PR, rétrospectives sur les échecs agents, les frictions setup et les instructions manquantes.

Phase 5 : Mesurer et décider du scale (*fin du pilot, puis mensuel*). Publier une bibliothèque de référence de skills, des exemples d’agents, des overlays par

projet et des tools déterministes seulement lorsque les patterns ont prouvé leur valeur. Former PM, CSM, QA, design. Mettre en place des code owners forts. Renforcer la CI/CD. Donner aux non-développeurs un environnement de contribution adapté, guidé et visible par l'équipe. Auditer les skills, détecter le drift, mesurer l'usage, supprimer l'obsolète et maintenir la documentation fiable. Le rôle de la gouvernance est d'informer, guider, aider et supporter, pas de dicter une manière unique de travailler.

Les pièges principaux sont connus. Chercher le prompt parfait dès le départ. Croire que le temps va s'accélérer tout seul. Mélanger deadlines fonctionnelles et expérimentation méthodologique. Laisser la documentation cachée là où elle est. Sous-estimer les contraintes d'environnement. Confondre adoption d'un outil et transformation d'une méthode.

Et il faut investir du temps réel : deux mois est le strict minimum pour transformer durablement la méthode de travail d'une équipe.

Roadmap synthétique

Phase	Durée indicative	Objectif	Livrable
Phase 0	1-2 semaines	Aligner le management	Objectif dominant explicite et attentes non basées sur une accélération immédiate.
Phase 1	2-4 semaines	Préparer le terrain	Environnement local, CI/CD, PR envs, documentation, tests, accès, repositories alignés.
Phase 2	2-4 semaines + continu	Onboarder l'agent	Instructions minimales, agents de base, tools déterministes, guardrails, formation non-dev, exemples ai-artifacts à adapter.

Phase	Durée indicative	Objectif	Livrable
Phase 3	6-8 semaines	Piloter	1 à 2 équipes d'early adopters (~7 ETP), cas faibles risques, support externe embarqué, validation fonctionnelle et technique explicites.
Phase 4	Pendant le pilot	Adapter les rituels	Standup de régulation, refinement plus court, démos PR env, rétrospectives setup.
Phase 5	Fin pilot + mensuel	Mesurer et scaler	Telemetry, audit, bibliothèque référence, formation transversale, support aux équipes.

Actions concrètes

1. Commencer par un setup réel avec une personne réelle sur un projet réel.
2. Clarifier l'objectif dominant du pilot avant de commencer.
3. Mettre en place une gouvernance minimale d'aide et de référence avant le passage à l'échelle.

15. Conclusion : l'avenir appartient aux orchestrateurs

Le 10x engineer est mort. Longue vie à l'orchestrateur.

Le développeur exceptionnel de demain ne sera pas seulement celui qui écrit le meilleur code le plus vite. Ce sera celui qui sait définir le bon problème, donner le bon contexte, choisir ce qui doit être délégué, superviser plusieurs agents, relire avec discernement, tester intelligemment, assumer le merge, apprendre vite et garder une vue d'ensemble.

L'orchestrateur n'est pas un manager bureaucratique de machines. C'est un ingénieur augmenté, un PM augmenté, un QA augmenté, un designer augmenté. C'est quelqu'un qui comprend que la valeur s'est déplacée de l'exécution pure vers la direction du système d'exécution.

Adopter l'IA, c'est gagner sur tous les tableaux si l'on fait le travail sérieusement. On apprend sur soi, parce que l'IA révèle nos habitudes, nos angles morts et nos peurs. On apprend sur les autres, parce que les rôles se rapprochent et que la collaboration devient plus explicite. On apprend une nouvelle manière de travailler. On se rend compte que beaucoup de choses que l'on acceptait comme normales étaient simplement mal organisées.

Et si rien ne marche ? Si l'IA décevait ? Si les agents étaient abandonnés ? Il resterait une pipeline CI/CD plus solide, une documentation plus accessible, des ingénieurs formés à plus de technologies, des équipes plus autonomes, une information produite mieux structurée, des workflows plus explicites, des managers plus proches du terrain.

Ce serait déjà une transformation utile.

Mais si cela marche, le changement est beaucoup plus profond. Le logiciel devient plus malléable. La connaissance fonctionnelle devient exécutable. Les rôles se recomposent. Les équipes livrent autrement. Les managers doivent apprendre autrement. Et les organisations qui auront investi tôt dans le contexte, la documentation de référence, le support aux équipes et la responsabilité auront un avantage difficile à rattraper.

La mort du code manuel n'est pas la mort de l'ingénieur.

C'est la fin d'une définition trop étroite de son métier.

Références

1. AWS DevOps & Developer Productivity Blog, **AI-Driven Development Life Cycle: Reimagining Software Engineering**, Raja SP, 31 July 2025. <https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/>
2. Microsoft .NET Blog, **Ten Months with Copilot Coding Agent in dotnet/runtime**, Stephen Toub, March 2026. Success story documentant le déploiement à grande échelle sur un projet avec des centaines de contributeurs : 878 PRs, 67.9% merge rate. <https://devblogs.microsoft.com/dotnet/ten-months-with-cca-in-dotnet-runtime/>
3. Microsoft, **HVE Core**, repository utilisé comme source upstream pour des prompts RPI dans TSF. <https://github.com/microsoft/hve-core>
4. Monorepo Storefront utilisé comme projet test pour le framework **ai-artifacts**, qui versionne, audite et compose agents, skills, tools, overlays et bases de connaissances réutilisables. <https://github.com/amadeus-nexwave/discovery-travelstorefront-monorepo>